Advanced Data Structures
CS4580

Lecture 08

Dictionaries

- Collection of items.
- Each item is a pair.
  - (key, element)
  - Pairs have different keys.

Application

- Collection of student records in this class.
  - (key, element) = (student name, linear list of assignment and exam scores)
  - All keys are distinct.
- Collection of in-use domain names.
  - (godaddy.com, owner information)
  - All keys are distinct.

Dictionary With Duplicates

- Keys are not required to be distinct.
- Word dictionary.
  - Items/pairs are of the form (word, meaning).
  - May have two or more entries for the same word.
    - (bolt, a threaded pin)
    - (bolt, a crash of thunder)
    - (bolt, to shoot forth suddenly)
    - (bolt, a gulp)
    - (bolt, a standard roll of cloth)
    - etc.

Dictionary Operations

- Static Dictionary.
  - initialize/create
  - get(theKey) (a.k.a. search)
  - CD ROM word dictionary
  - CD ROM geographic database of cities, rivers, roads, auto navigation system, etc.
- Dynamic Dictionary.
  - get(theKey) (a.k.a. search)
  - put(theKey, theElement) (a.k.a. insert)
  - remove(theKey) (a.k.a. delete)

Hash Table Dictionaries

- O(1) expected time for get, put, and remove.
- O(n) worst-case time for get, put, and remove.
  - O(log n) if overflows handled by balanced search trees.
- Not suitable for nearest match queries.
  - Get element with smallest key >= theKey.
- Not suitable for range queries.
- Not suitable for indexed operations.
  - Get element with third smallest key.
  - Remove element with 5th smallest key.

Bin Packing

- n items to be packed into bins
- each item has a size
- each bin has a capacity of c
- minimize number of bins

Bin Packing Heuristics

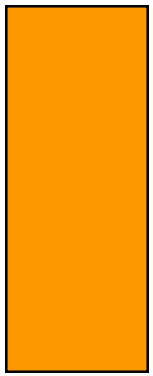
- Best Fit.
  - Items are packed one at a time in given order.
  - To determine the bin for an item, first determine set S of bins into which the item fits.
  - If S is empty, then start a new bin and put item into this new bin.
  - Otherwise, pack into bin of S that has least available capacity.

Best Fit Example

n = 4

weights = **[4, 7, 3, 6]**

capacity = 10



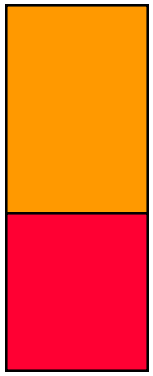
Pack red item into first bin.

Best Fit

n = 4

weights = **[4, 7, 3, 6]**

capacity = 10



Pack blue item next.

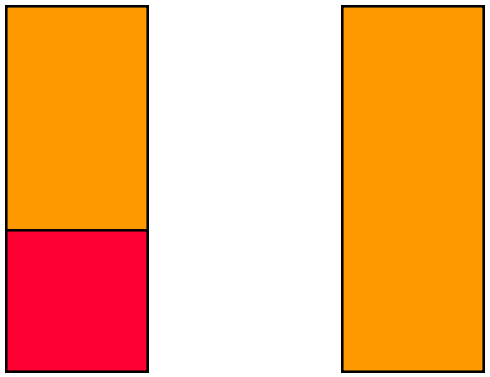
Doesn’t fit, so start a new bin.

Best Fit

n = 4

weights = [4, 7, 3, 6]

capacity = 10

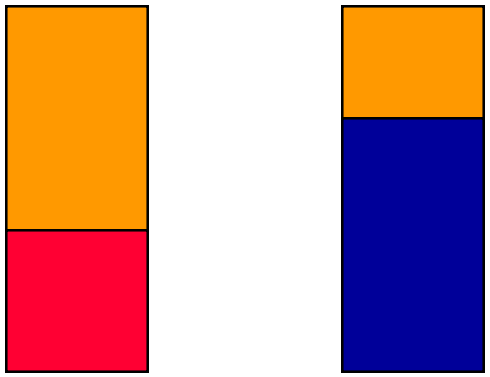


Best Fit

n = 4

weights = **[4, 7, 3, 6]**

capacity = 10



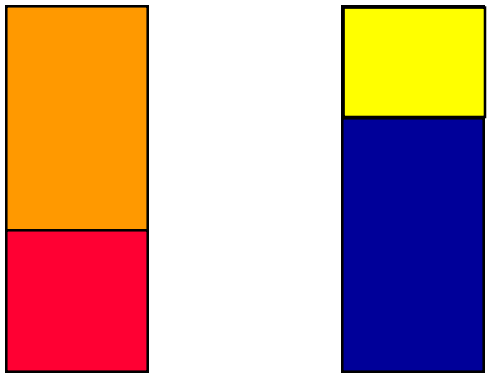
Pack yellow item into second bin.

Best Fit

n = 4

weights = **[4, 7, 3, 6]**

capacity = 10



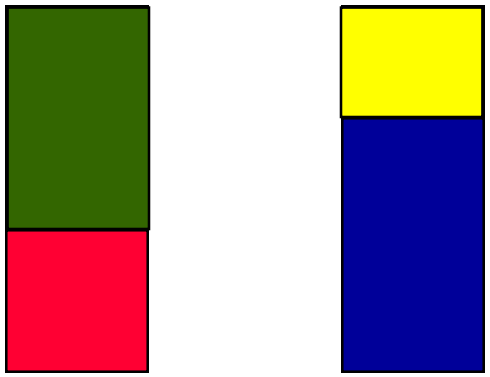
Pack green item into first bin.

Best Fit

n = 4

weights = **[4, 7, 3, 6]**

capacity = 10

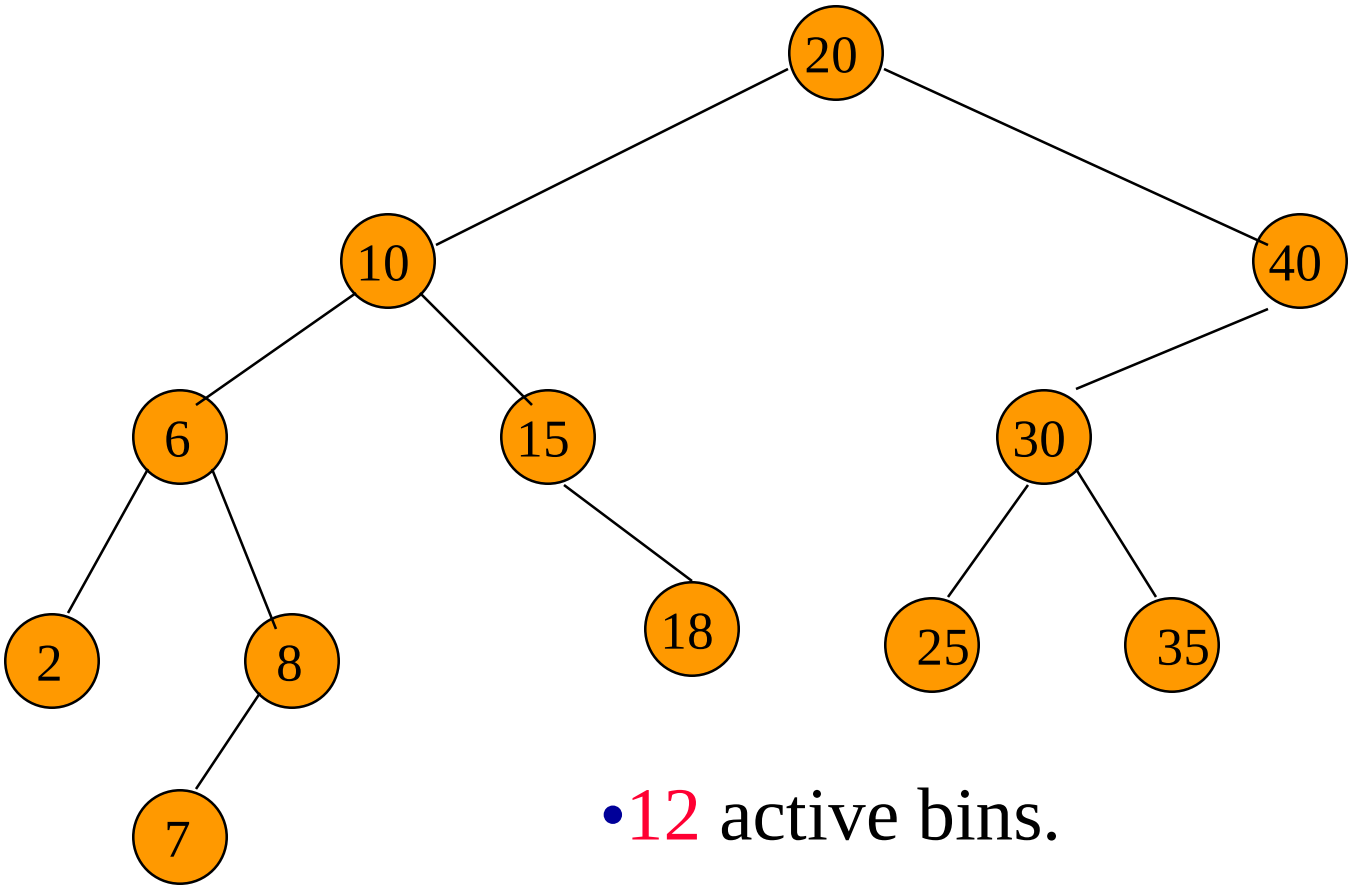


Optimal packing.

Implementation Of Best Fit

- Use a dynamic dictionary (with duplicates) in which the items are of the form (available capacity, bin index).
- Pack an item whose requirement is s.
  - Find a bin with smallest available capacity >= s.
  - Reduce available capacity of this bin by s.
    - May be done by removing old pair and inserting new one.
  - If no such bin, start a new bin.
    - Insert a new pair into the dictionary.

Best Fit Example



- 12 active bins.
- Pack item whose size is 22.

Complexity Of Best Fit

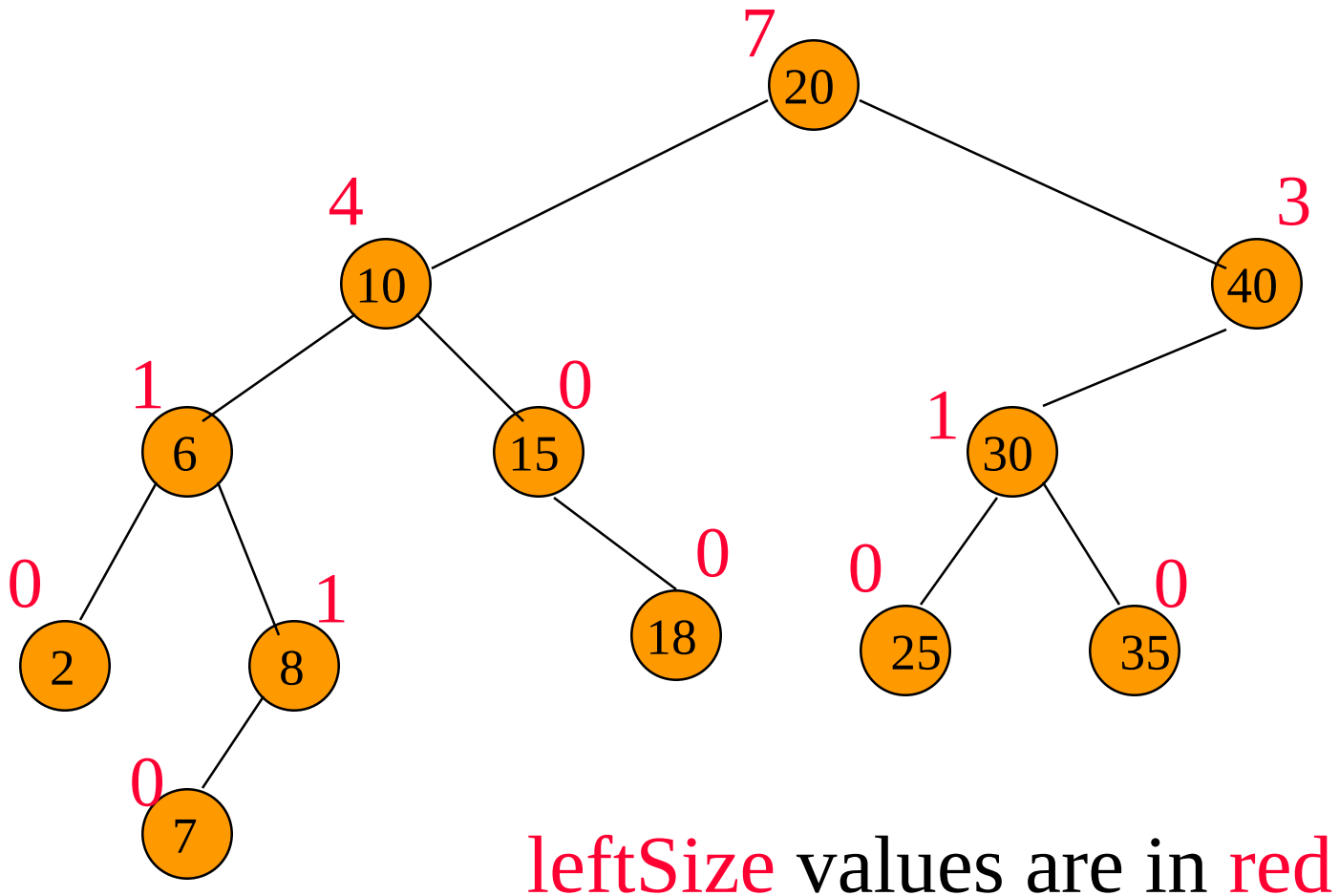


- Use a balanced binary search tree (with duplicates) in which the pairs are (available capacity, bin index).
- O(n) get, put, and remove/put operations, where n is the number of items to be packed.
- O(n log n).

Indexed Binary Search Tree

- Binary search tree.
- Each node has an additional field.
  - leftSize = number of nodes in its left subtree

Example Indexed Binary Search Tree



leftSize values are in red

leftSize And Rank

Rank of an element is its position in inorder (inorder = ascending key order).

[2,6,7,8,10,15,18,20,25,30,35,40]

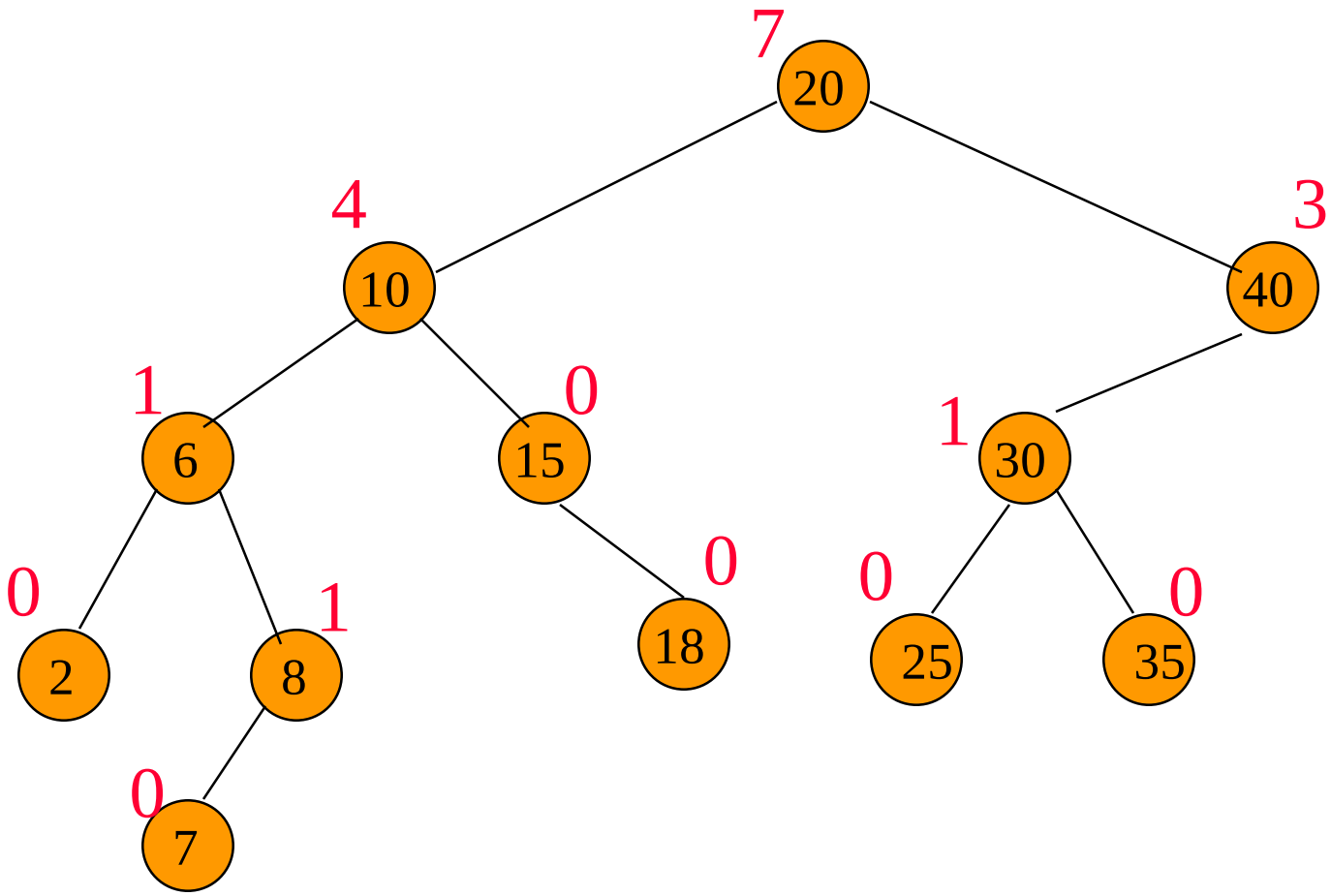
rank(2) = 0

rank(15) = 5

rank(20) = 7

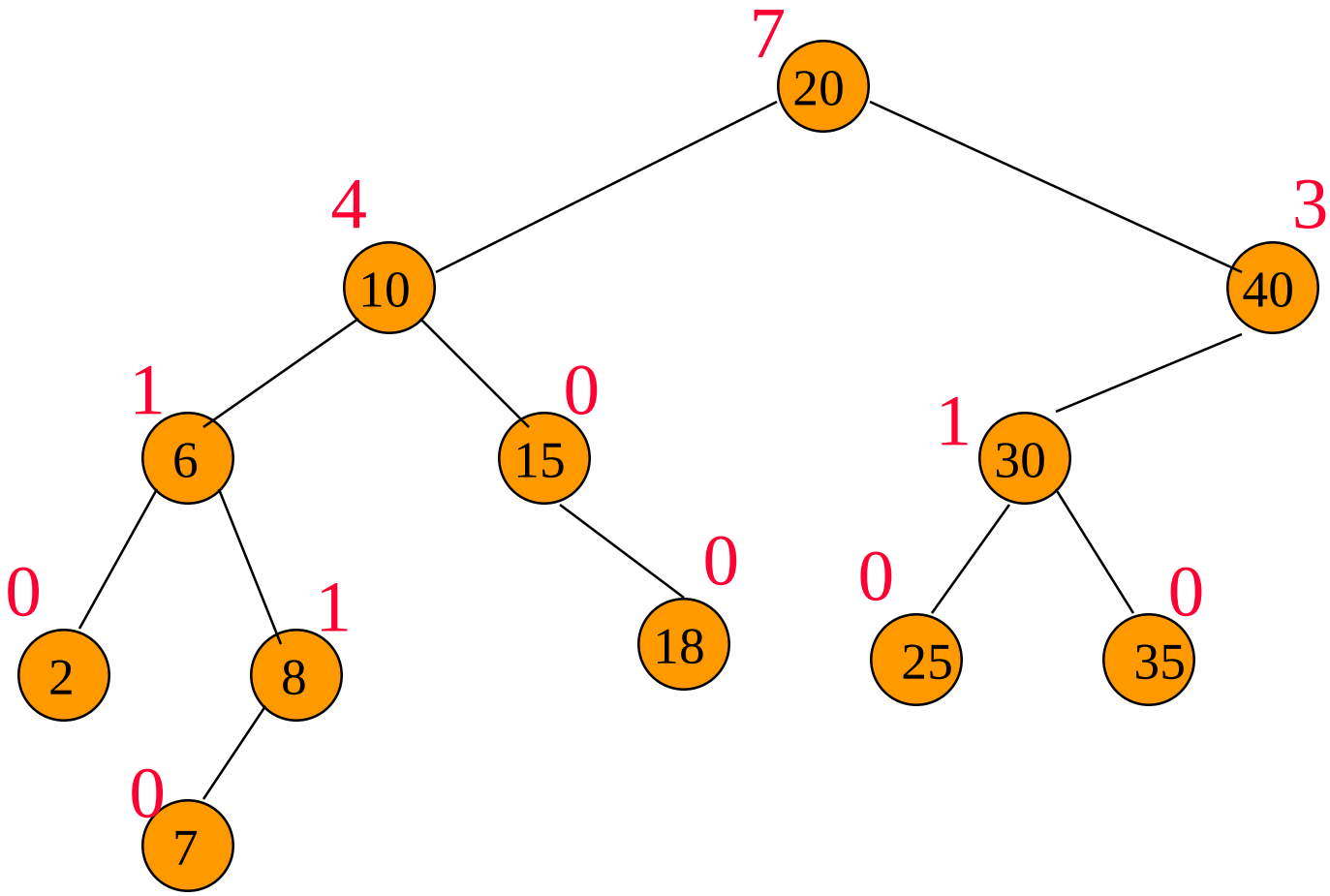
lextSize(x) = rank(x) with respect to elements in subtree rooted at x

leftSize And Rank



sorted list = [2,6,7,8,10,15,18,20,25,30,35,40]

get(index)

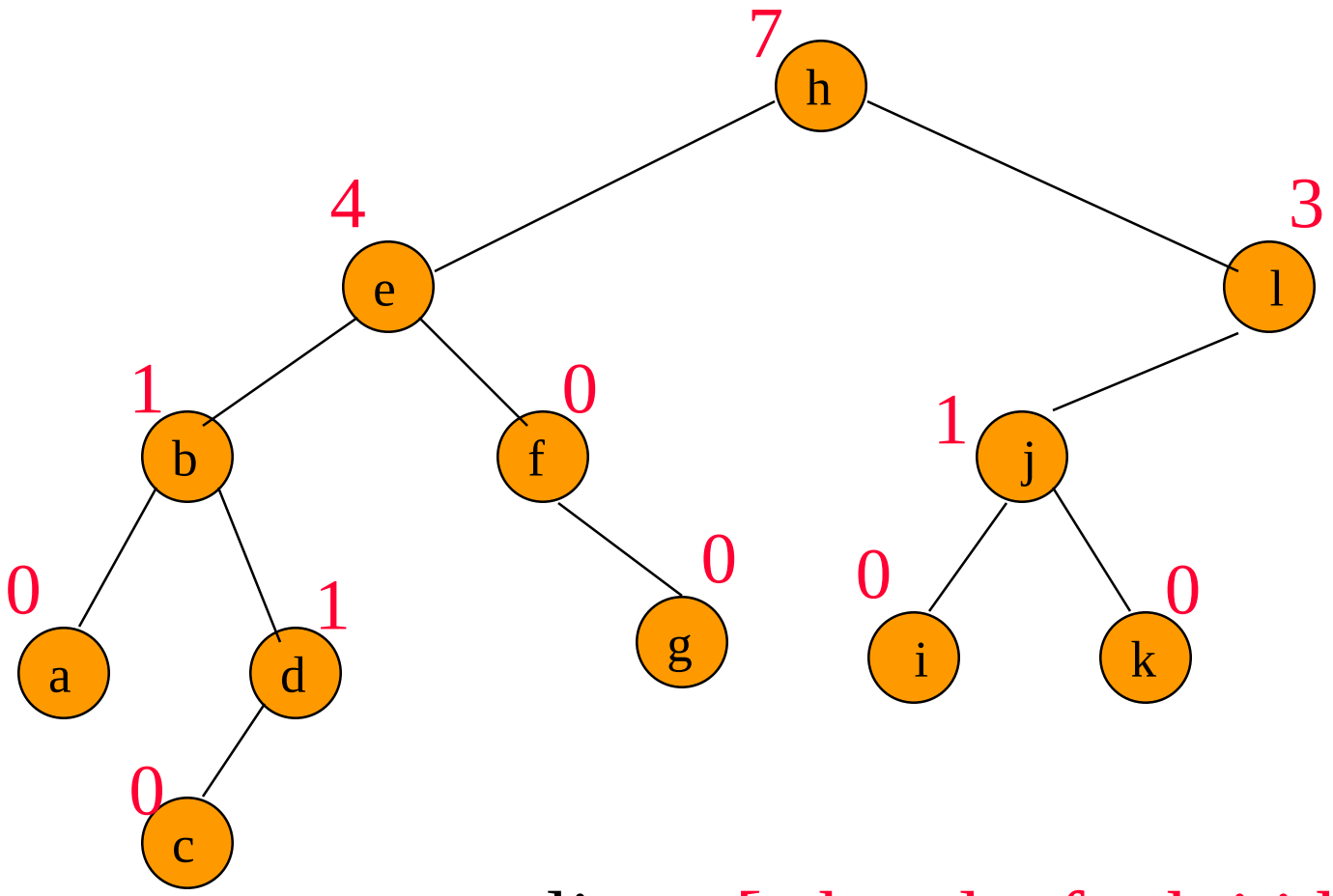


sorted list = [2,6,7,8,10,15,18,20,25,30,35,40]

get(index)

- if index = x.leftSize desired element is x.element
- if index < x.leftSize desired element is index'th element in left subtree of x
- if index > x.leftSize desired element is (index – x.leftSize – 1)'th element in right subtree of x

Linear List As Indexed Binary Tree



list = [a,b,c,d,e,f,g,h,i,j,k,l]

Performance

- Linear List.
  - get(index)
  - put(index, element)
  - remove(index)
- Array.
  - O(1), O(n), O(n).
- Chain.
  - O(n), O(n), O(n).
- Indexed AVL Tree (IAVL)
  - O(log n), O(log n), O(log n).

Experimental Results

- 40,000 of each operation.
- Java code on a 350MHz PC

Performance

Indexed AVL Tree (IAVL)

Operation	Array	Chain	IAVL
get	5.6ms	157sec	63ms
average puts	5.8sec	115sec	392ms
worst-case puts	11.8sec	157sec	544ms
average removes	5.8sec	149sec	1.5sec
worst-case removes	11.7sec	157sec	1.6sec

Time for 40,000 operations

Focus

- Tree structures for static and dynamic dictionaries.

Static Dictionaries

- Collection of items.
- Each item is a pair.
  - (key, element)
  - Pairs have different keys.
- Operations are:
  - initialize/create
  - get (search)

Hashing

- Perfect hashing (no collisions).
- Minimal perfect hashing (space = n).
- CHD algorithm.
- O(n) time to construct the perfect or minimal perfect hash function.
- O(1) search time.
- Bothelo, Belazzougui & Dietzfelbinger. Compress, hash, and displace. *17th European Symposium on Algorithms, 2009*.

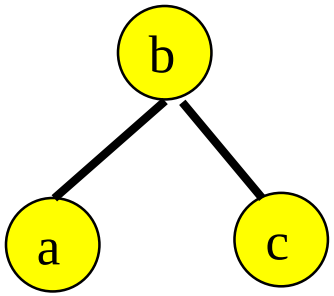
Search Tree

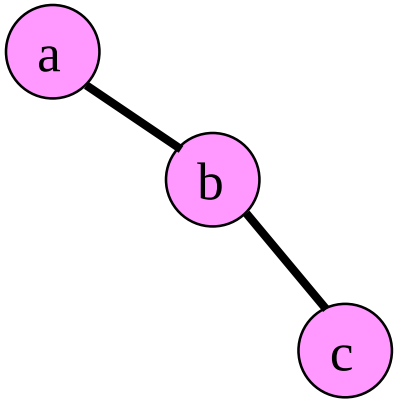
- Hashing not efficient for extended operations such as range search and nearest match.
- Will examine a binary search tree structure for static dictionaries.
- Each item/key/element has an estimated access frequency (or probability).

Example

Key	Probability
a	0.8
b	0.1
c	0.1

$a < b < c$





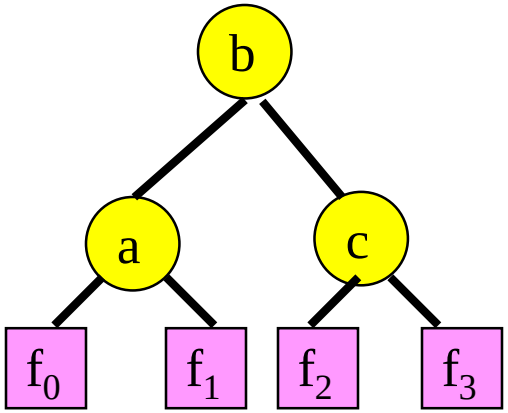
Cost = 0.8 * 2 + 0.1 * 1 + 0.1 * 2

= 1.9

Cost = 0.8 * 1 + 0.1 * 2 + 0.1 * 3

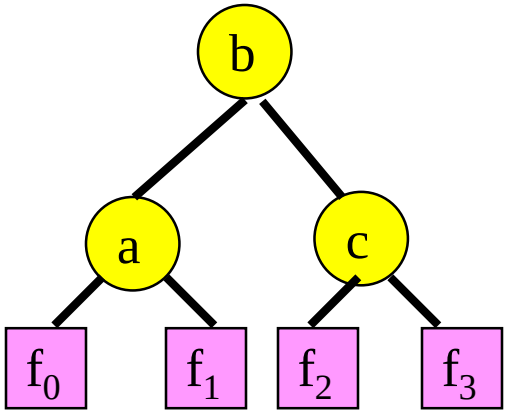
= 1.3

Search Types



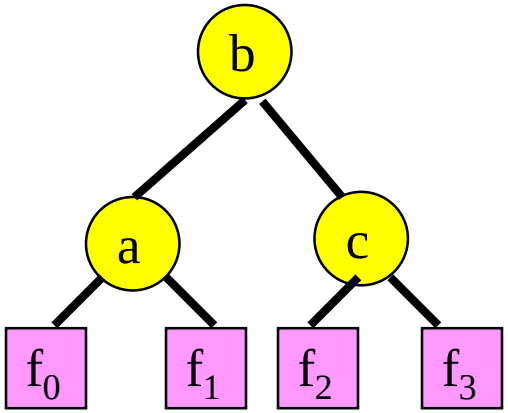
- Successful.
  - Search for a key that is in the dictionary.
  - Terminates at an internal node.
- Unsuccessful.
  - Search for a key that is not in the dictionary.
  - Terminates at an external/failure node.

Internal And External Nodes



- A binary tree with n internal nodes has $n + 1$ external nodes.
- Let s_1, s_2, …, s_n be the internal nodes, in inorder.
- $key(s_1) < key(s_2) < \ldots < key(s_n)$.
- Let $key(s_0) = -infinity$ and $key(s_{n+1}) = infinity$.
- Let f_0, f_1, …, f_n be the external nodes, in inorder.
- f_i is reached iff $key(s_i) < search\ key < key(s_{i+1})$.

Cost Of Binary Search Tree



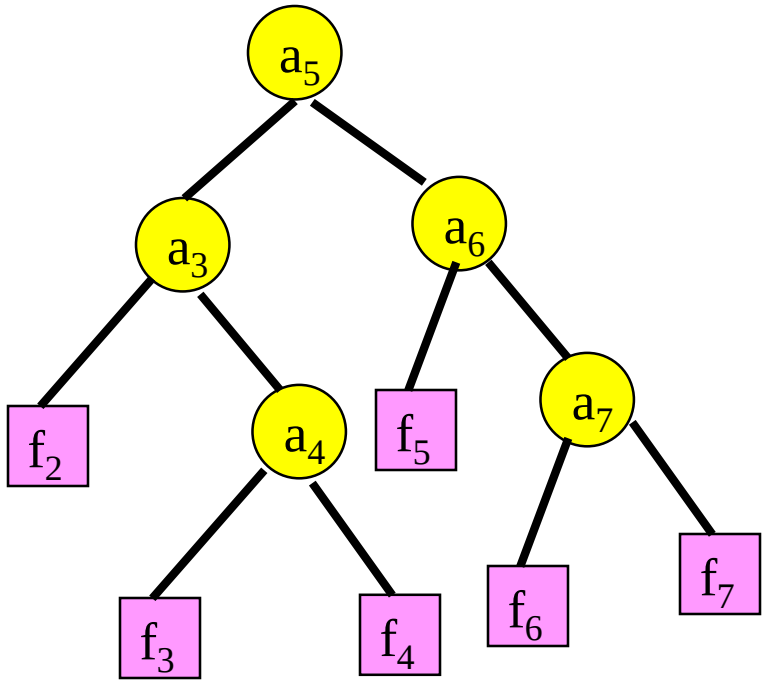
- Let p_i = probability for $key(s_i)$.
- Let q_i = probability for $key(s_i) < \text{search key} < key(s_{i+1})$.
- Sum of ps and qs = 1.
- Cost of tree = $\Sigma_{0 <= i <= n} q_i \, (level(f_i) - 1)$
  $+ \Sigma_{1 <= i <= n} p_i \, * level(s_i)$
- Cost = weighted path length.

Brute Force Algorithm

- Generate all binary search trees with n internal nodes.
- Compute the weighted path length of each.
- Determine tree with minimum weighted path length.
- Number of trees to examine is $O(4^n/n^{1.5})$.
- Brute force approach is impractical for large n.

Dynamic Programming

- Keys are $a_1 < a_2 < \ldots < a_n$.
- Let $T_{i\,j}$= least cost tree for $a_{i+1}, a_{i+2}, \ldots, a_j$.
- T_{0n}= least cost tree for $a_1, a_2, \ldots, a_n$.



$T_{2,7}$

- $T_{i\,j}$ includes $p_{i+1}, p_{i+2}, \ldots, p_j$ and $q_i, q_{i+1}, \ldots, q_j$.

Terminology

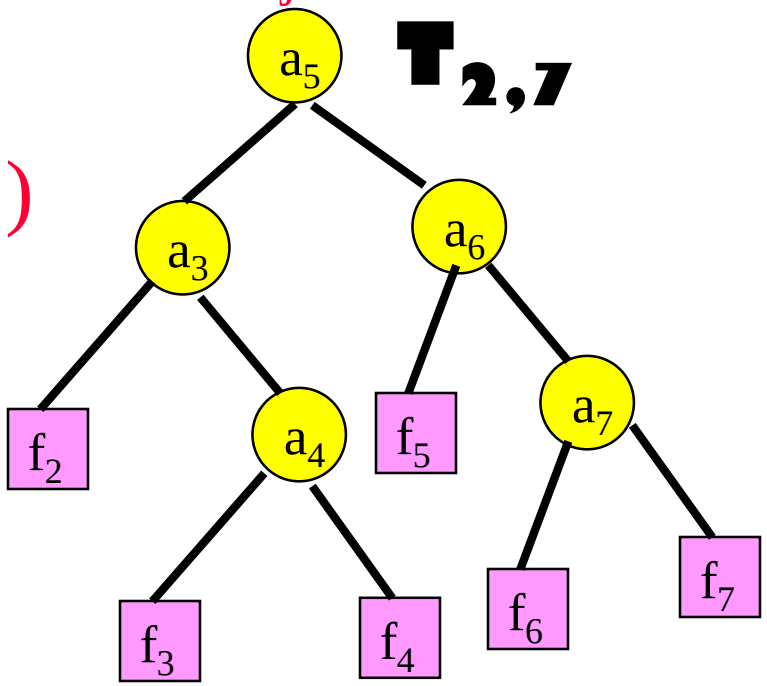
- $T_{i\,j}$= least cost tree for a_{i+1}, a_{i+2}, …, a_j.
- $c_{i\,j}$= cost of $T_{i\,j}$

  $= \Sigma_{i <= u <= j}\, q_u\, (level(f_u) - 1)$

  $+ \Sigma_{i < u <= j}\, p_u\, * level(s_u)$.
- $r_{i\,j}$= root of $T_{i\,j}$.
- $w_{i\,j}$= weight of $T_{i\,j}$

  = sum of ps and qs in $T_{i\,j}$

  $= p_{i+1} + p_{i+2} + \ldots + p_j + q_i + q_{i+1} + \ldots + q_j$



i = j

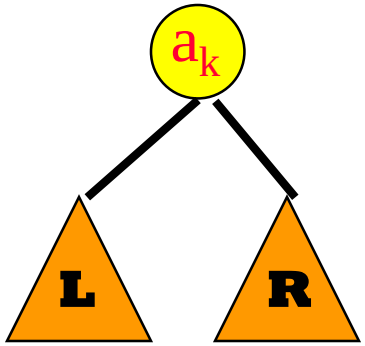
- $T_{i\,j}$ includes p_{i+1}, p_{i+2}, …, p_j and q_i, q_{i+1}, …, q_j.
- $T_{i\,i}$ includes q_i only.

f_i T_{ii}

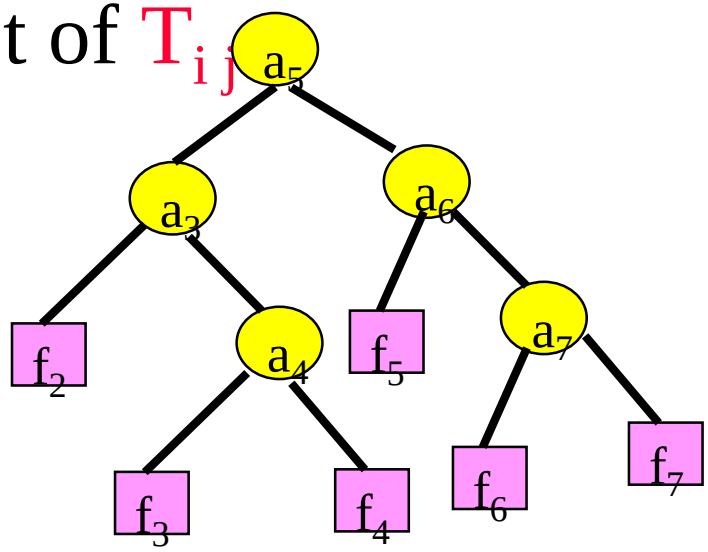
- $c_{i\,i}$ = cost of $T_{i\,i} = 0$.
- $r_{i\,i}$ = root of $T_{i\,i} = 0$.
- $w_{i\,i}$ = weight of $T_{i\,i}$
  = sum of ps and qs in $T_{i\,i}$
  = q_i

i < j

- $T_{i\,j}$= least cost tree for a_{i+1}, a_{i+2}, …, a_j.
- $T_{i\,j}$ includes p_{i+1}, p_{i+2}, …, p_j and q_i, q_{i+1}, …, q_j.
- Let a_k, $i < k <= j$, be in the root of $T_{i\,j}$

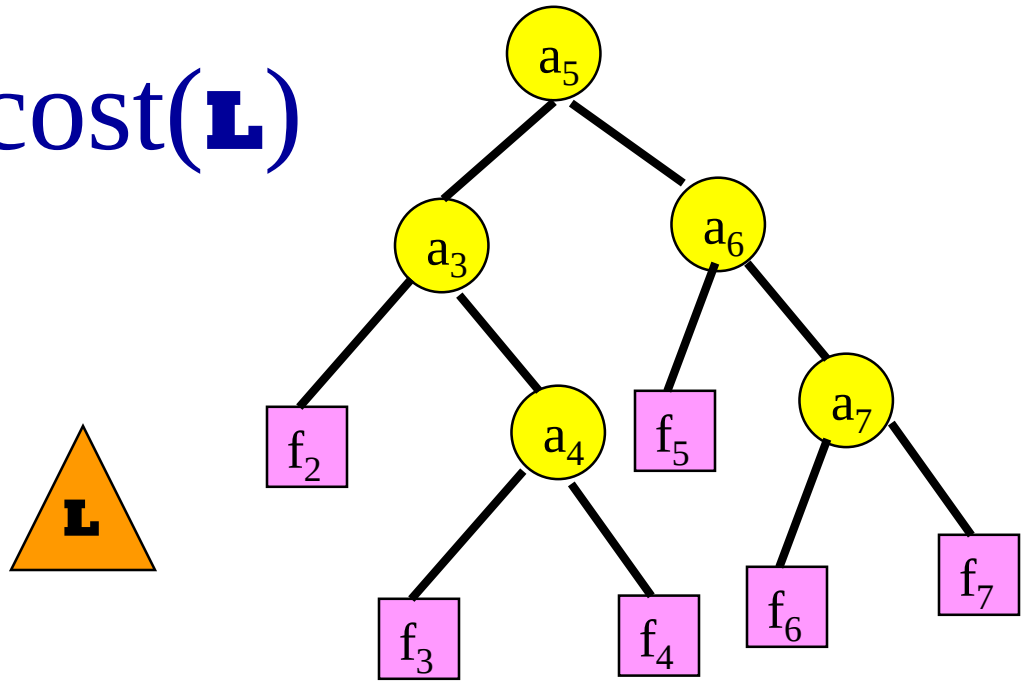


$T_{i\,j}$



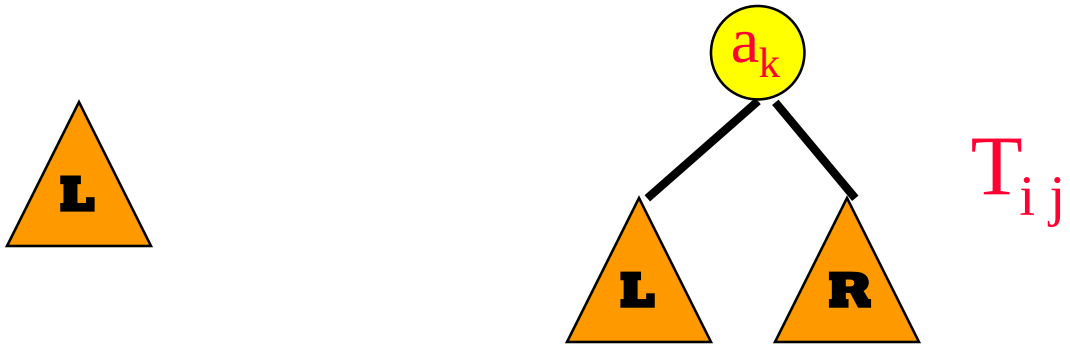
- **L** includes p_{i+1}, p_{i+2}, …, p_{k-1} and q_i, q_{i+1}, …, q_{k-1}.
- **R** includes p_{k+1}, p_{k+2}, …, p_j and q_k, q_{k+1}, …, q_j.

cost(**L**)

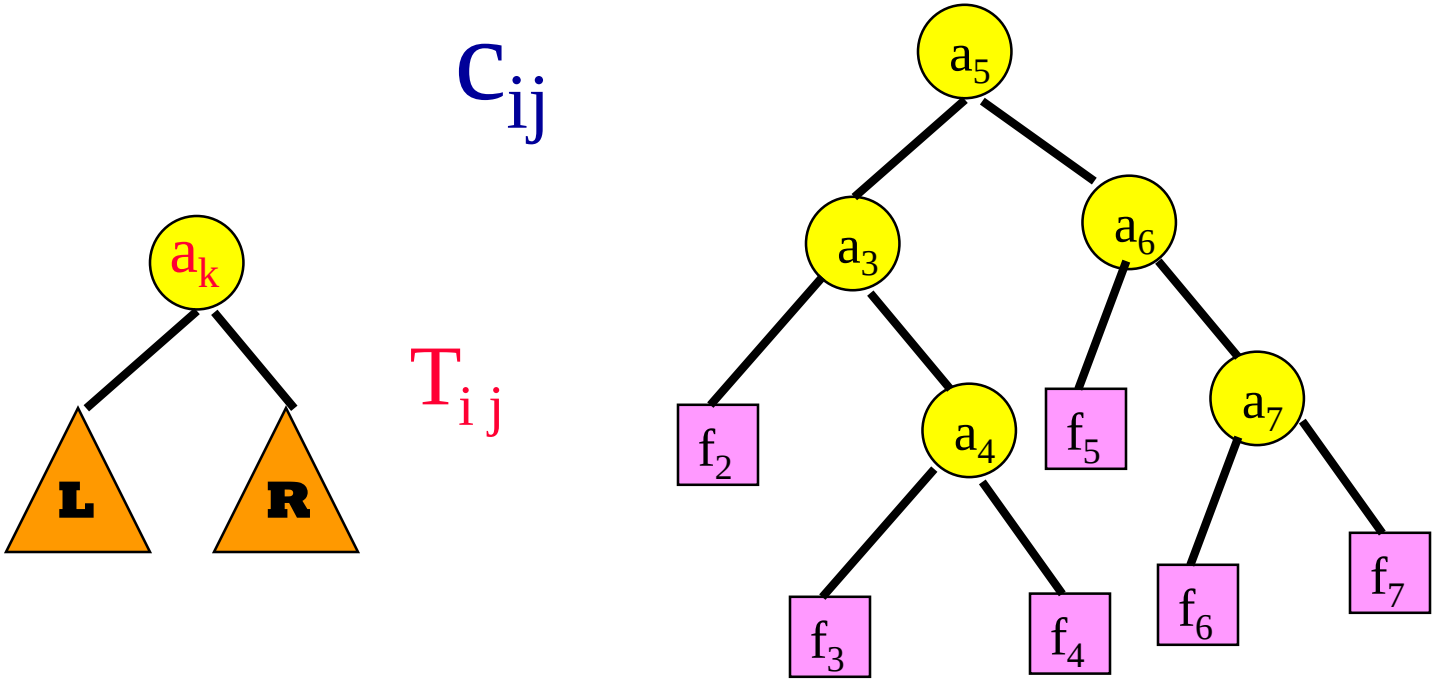


- **L** includes p_{i+1}, p_{i+2}, …, p_{k-1} and q_i, q_{i+1}, …, q_{k-1}.
- cost(**L**) = weighted path length of **L** when viewed as a stand alone binary search tree.

Contribution To c_{ij}



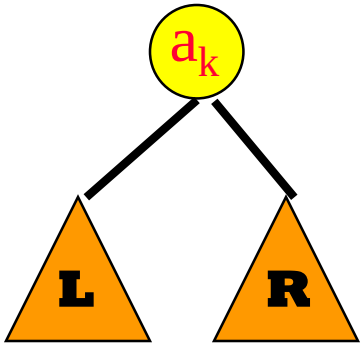
- $c_{i\,j} = \Sigma_{i <= u <= j}\, q_u\, (\text{level}(f_u) - 1)$
  $+ \Sigma_{i < u <= j}\, p_u\, * \text{level}(s_u)$.
- When **L** is viewed as a subtree of $T_{i\,j}$, the level of each node is 1 more than when **L** is viewed as a stand alone tree.
- So, contribution of **L** to c_{ij} is cost(**L**) + $w_{i\,k-1}$.



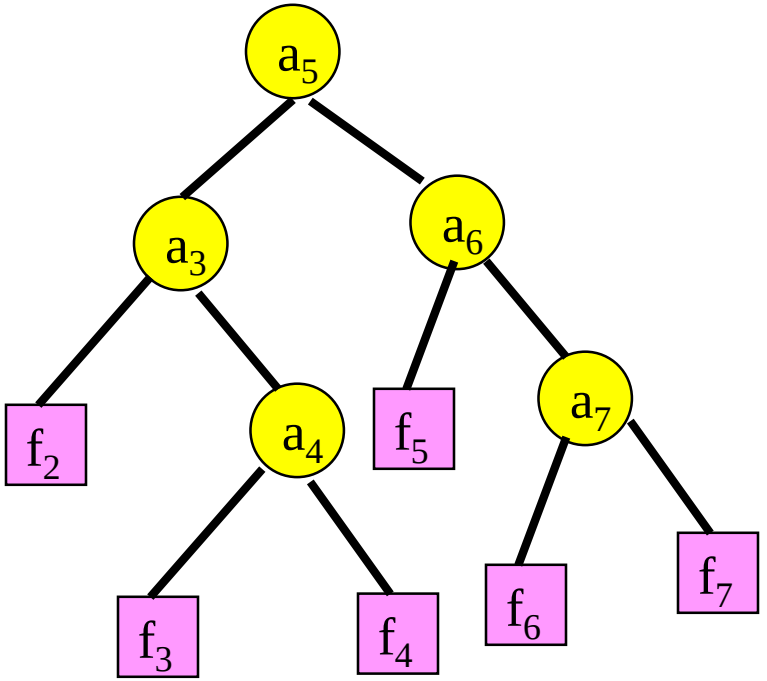
- Contribution of **L** to c_{ij} is cost(**L**) + $w_{i\,k-1}$.
- Contribution of **R** to c_{ij} is cost(**R**) + w_{kj}.
- c_{ij} = cost(**L**) + $w_{i\,k-1}$ + cost(**R**) + w_{kj} + p_k

  = cost(**L**) + cost(**R**) + w_{ij}

c_{ij}

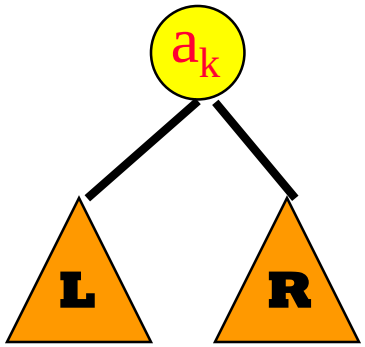


$T_{i\,j}$



- $c_{ij} = cost(\mathbf{L}) + cost(\mathbf{R}) + w_{ij}$
- $cost(\mathbf{L}) = c_{ik-1}$
- $cost(\mathbf{R}) = c_{kj}$
- $c_{ij} = c_{ik-1} + c_{kj} + w_{ij}$
- Don't know k.
- $c_{ij} = \min_{i < k <= j}\{c_{ik-1} + c_{kj}\} + w_{ij}$

c_{ij}



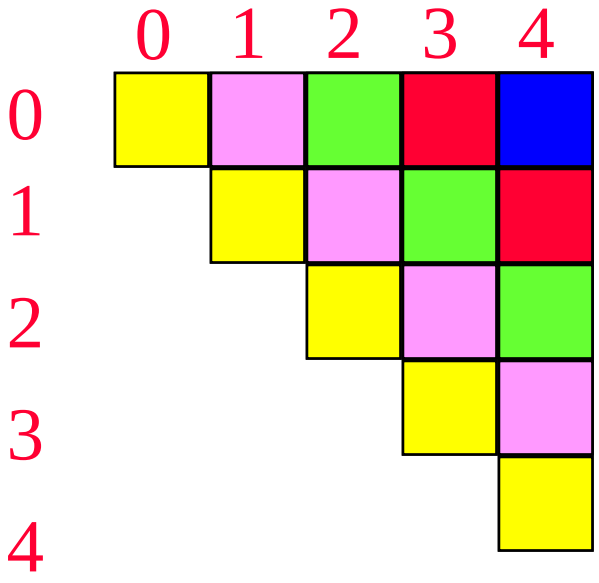
$T_{i\,j}$

- $c_{ij} = \min_{i < k <= j}\{c_{ik-1} + c_{kj}\} + w_{ij}$
- $r_{ij} = k$ that minimizes right side.

Computation Of c_{0n} And r_{0n}

- Start with $c_{i\,i} = 0$, $r_{i\,i} = 0$, $w_{i\,i} = q_i$, $0 <= i <= n$ (zero-key trees).
- Use $c_{ij} = \min_{i < k <= j}\{c_{ik-1} + c_{kj}\} + w_{ij}$ to compute c_{ii+1}, $r_{i\,i+1}$, $0 <= i <= n - 1$ (one-key trees).
- Now use the equation to compute c_{ii+2}, $r_{i\,i+2}$, $0 <= i <= n - 2$ (two-key trees).
- Now use the equation to compute c_{ii+3}, $r_{i\,i+3}$, $0 <= i <= n - 3$ (three-key trees).
- Continue until c_{0n} and r_{0n}(n-key tree) have been computed.

Computation Of c_{0n} And r_{0n}



c_{ij}, r_{ij}, i <= j

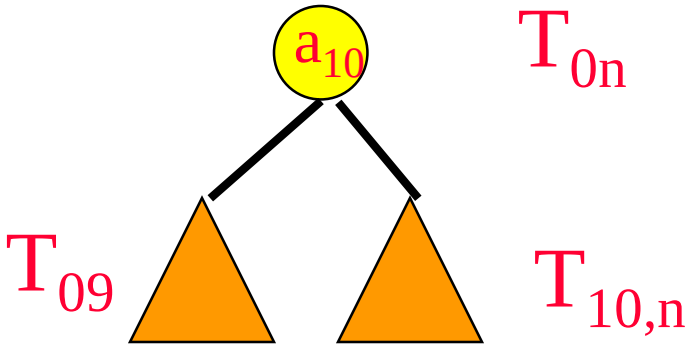
Complexity

- $c_{ij} = \min_{i < k <= j}\{c_{ik-1} + c_{kj}\} + w_{ij}$
- $O(n)$ time to compute one c_{ij}.
- $O(n^2)$ c_{ij}s to compute.
- Total time is $O(n^3)$.
- May be reduced to $O(n^2)$ by using

$$c_{ij} = \min_{ri,j-1 < k <= ri+1,j}\{c_{ik-1} + c_{kj}\} + w_{ij}$$

Construct T_{0n}

- Root is r_{0n}.
- Suppose that $r_{0n} = 10$.



- Construct T_{09} and $T_{10,n}$ recursively.
- Time is $O(n)$.

Dynamic Dictionaries

- Primary Operations:
  - get(key) => search
  - put(key, element) => insert
  - remove(key) => delete
- Additional operations:
  - ascend()
  - get(index)
  - remove(index)

Complexity Of Dictionary Operations

get(), put() and remove()

Data Structure	Worst Case	Expected
Hash Table	O(n)	O(1)
Binary Search Tree	O(n)	O(log n)
Balanced Binary Search Tree	O(log n)	O(log n)

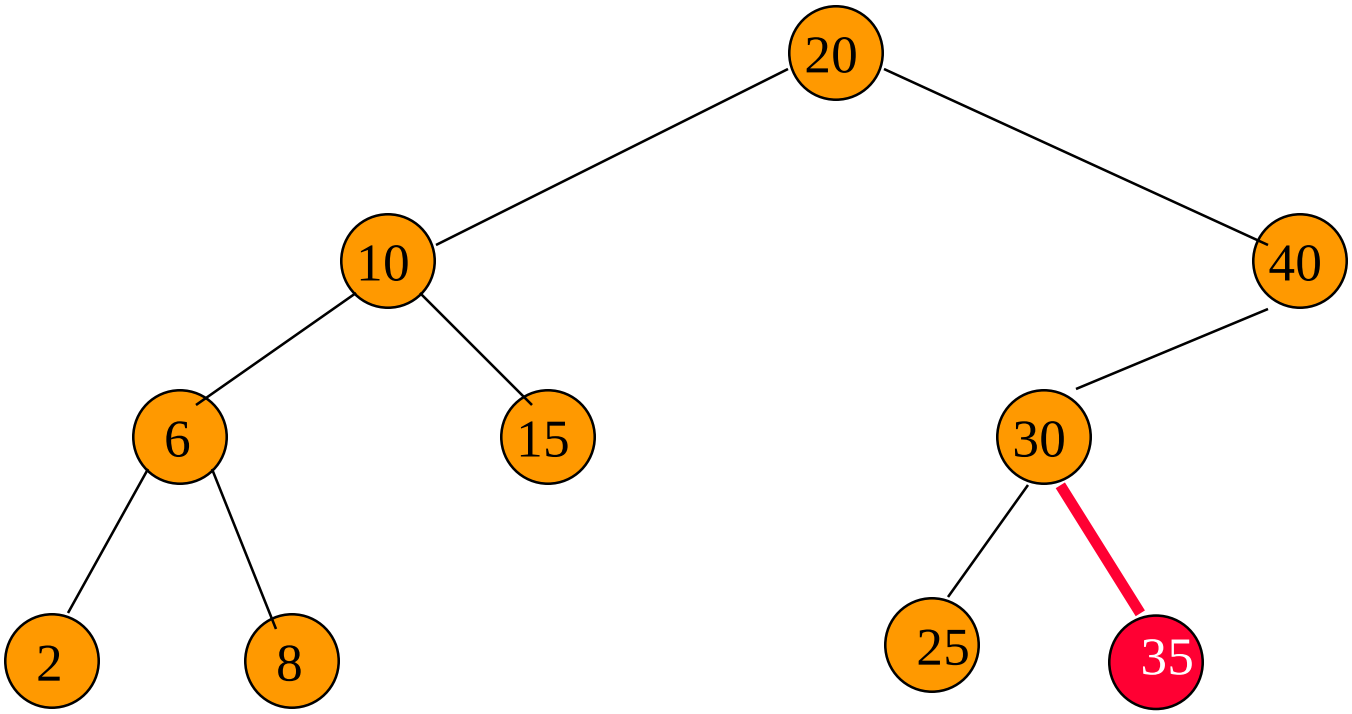
n is number of elements in dictionary

Complexity Of Other Operations
ascend(), get(index), remove(index)

Data Structure	ascend	get and remove
Hash Table	O(D + n log n)	O(D + n log n)
Indexed BST	O(n)	O(n)
Indexed Balanced BST	O(n)	O(log n)

D is number of buckets

The Operation put()



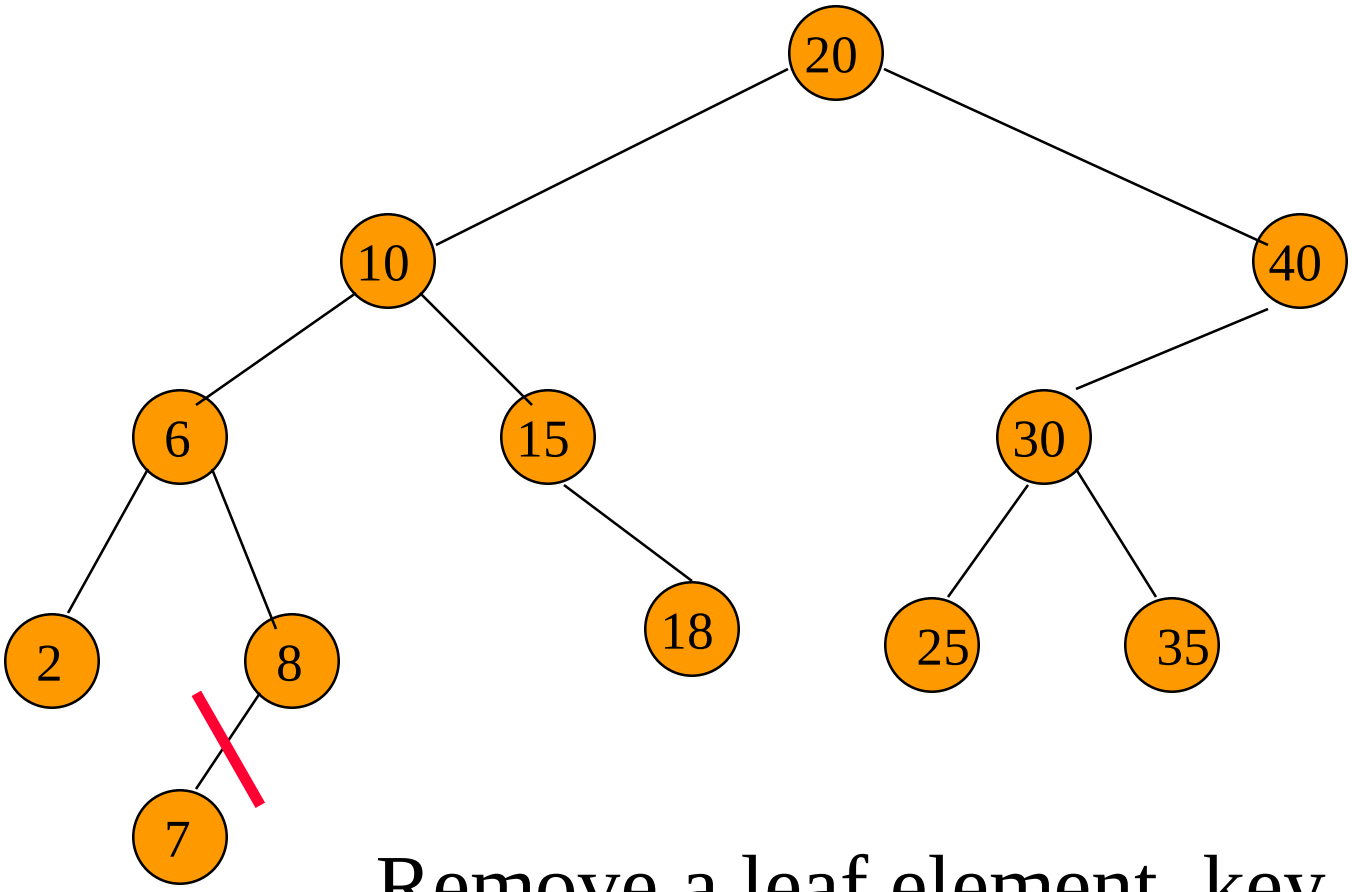
Put a pair whose key is 35.

The Operation remove()

Three cases:

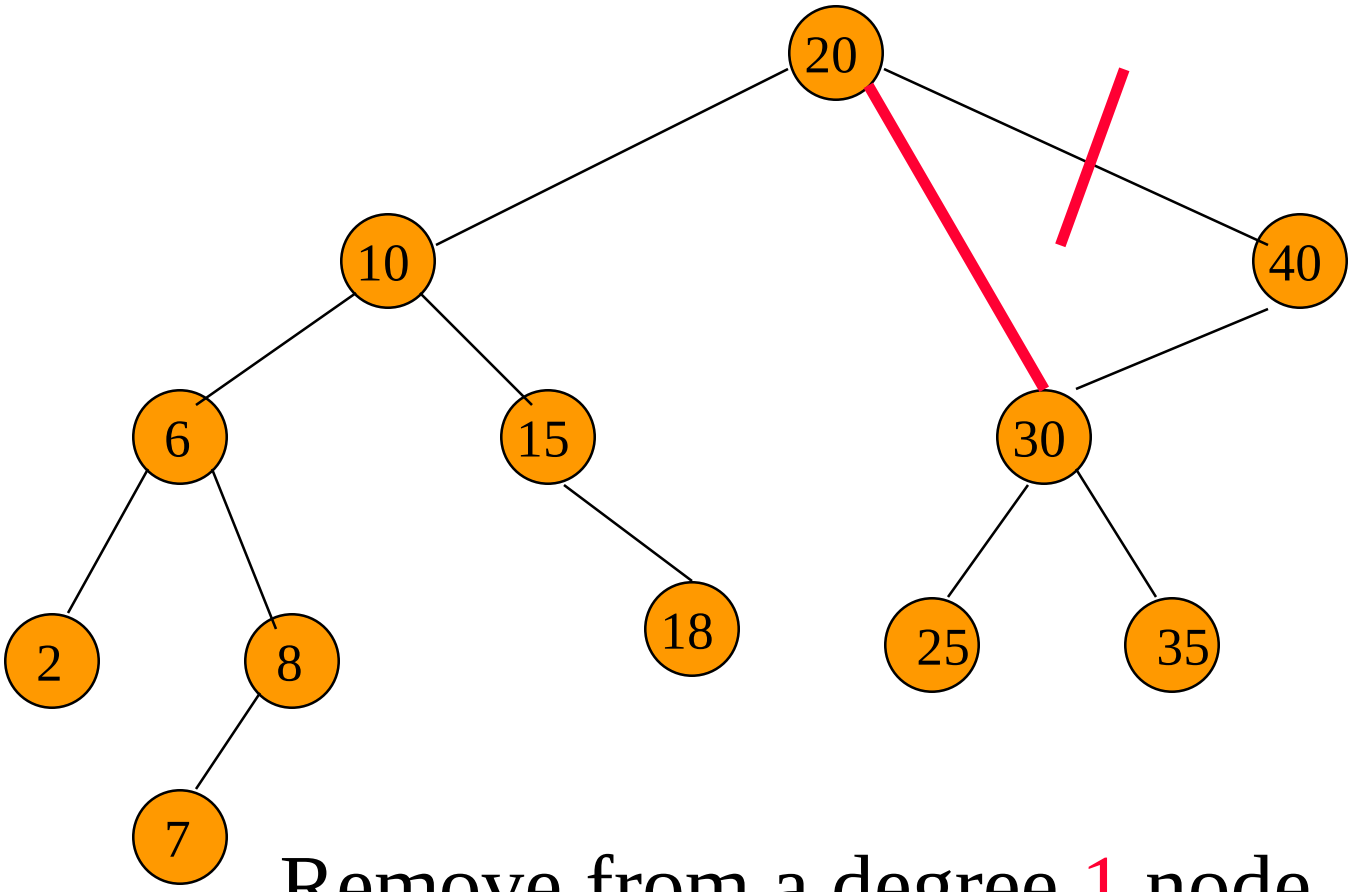
- Element is in a leaf.
- Element is in a degree 1 node.
- Element is in a degree 2 node.

Remove From A Leaf



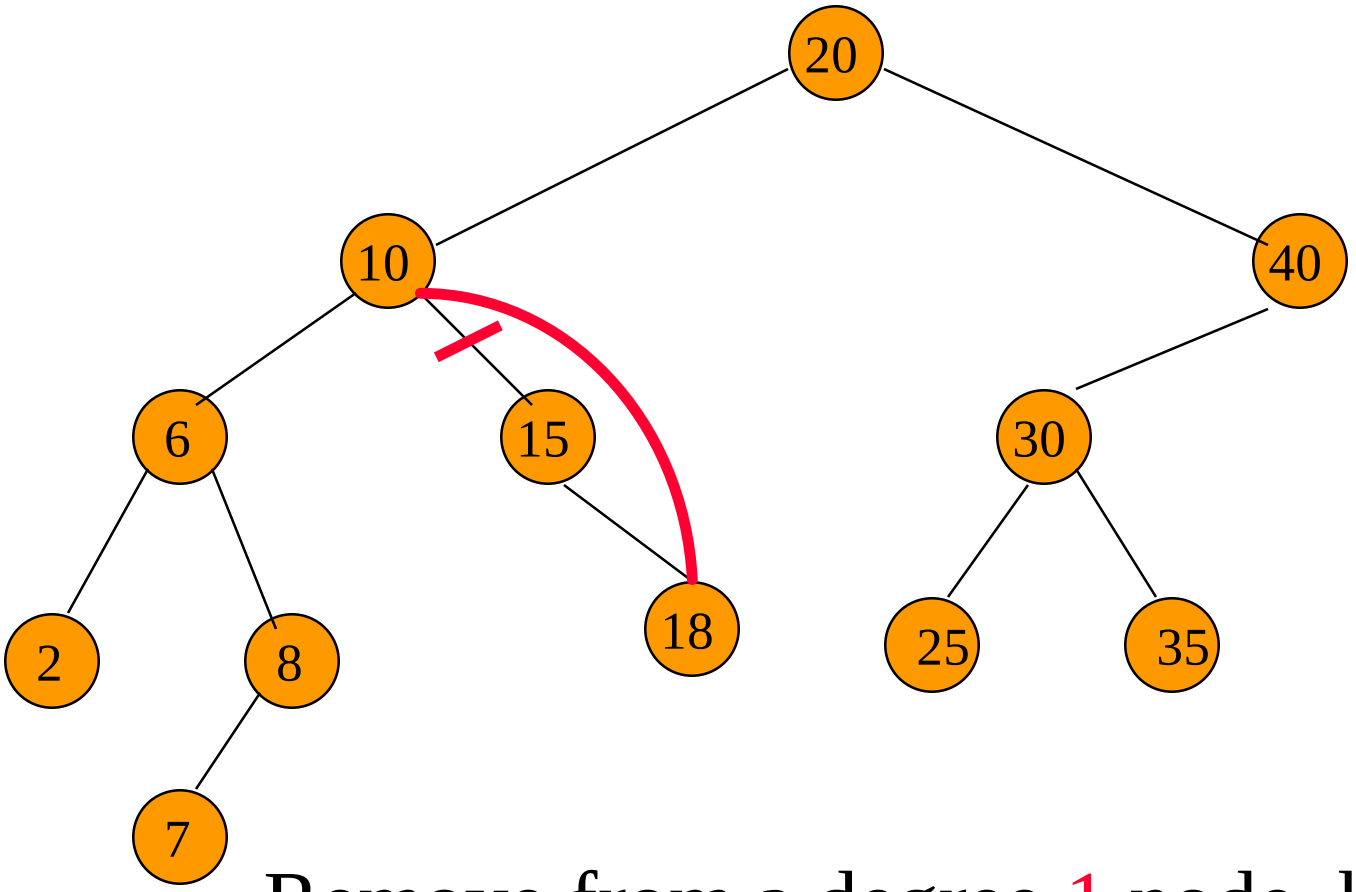
Remove a leaf element. key = 7

Remove From A Degree 1 Node



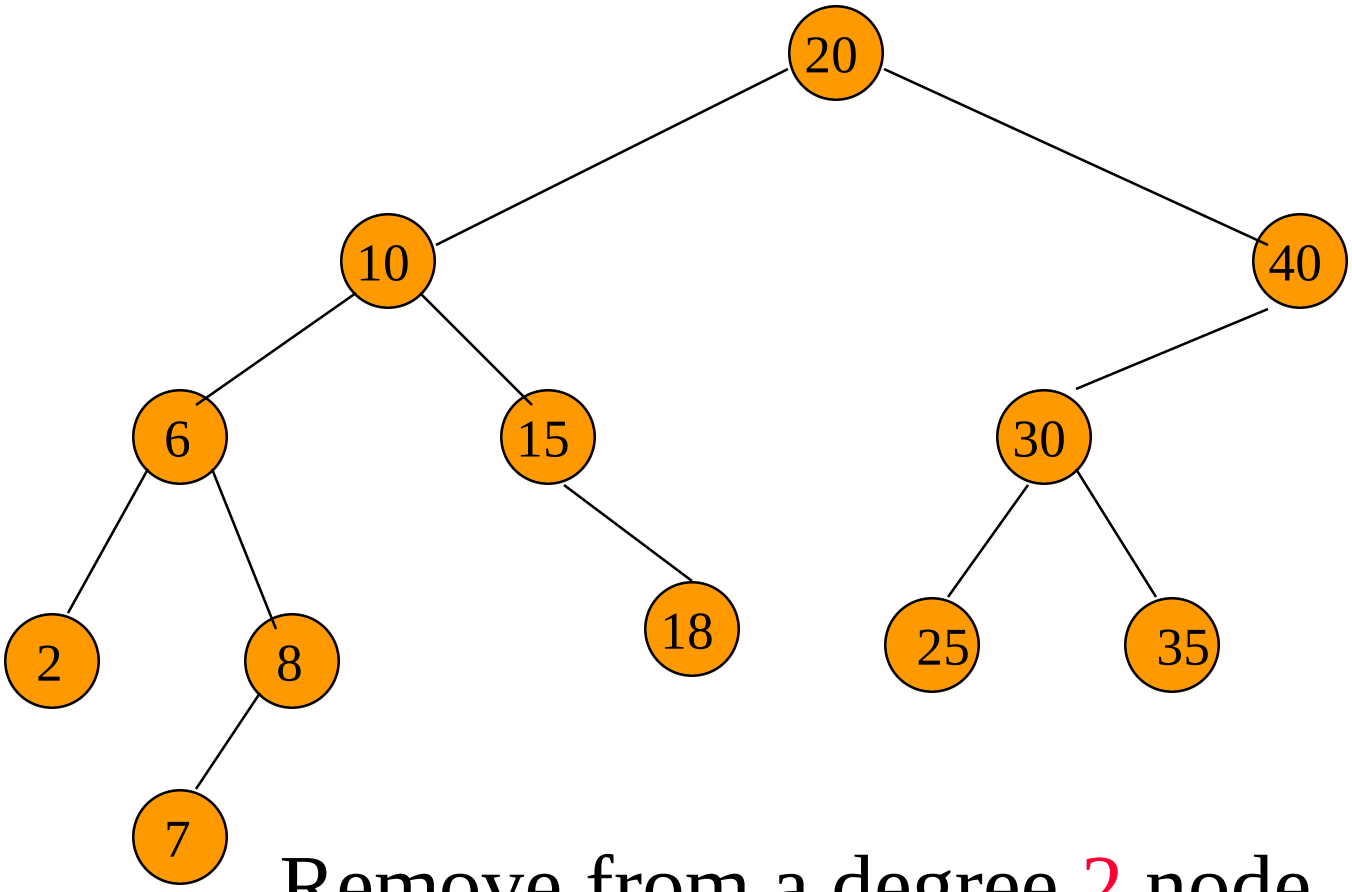
Remove from a degree 1 node. key = 40

Remove From A Degree 1 Node (contd.)



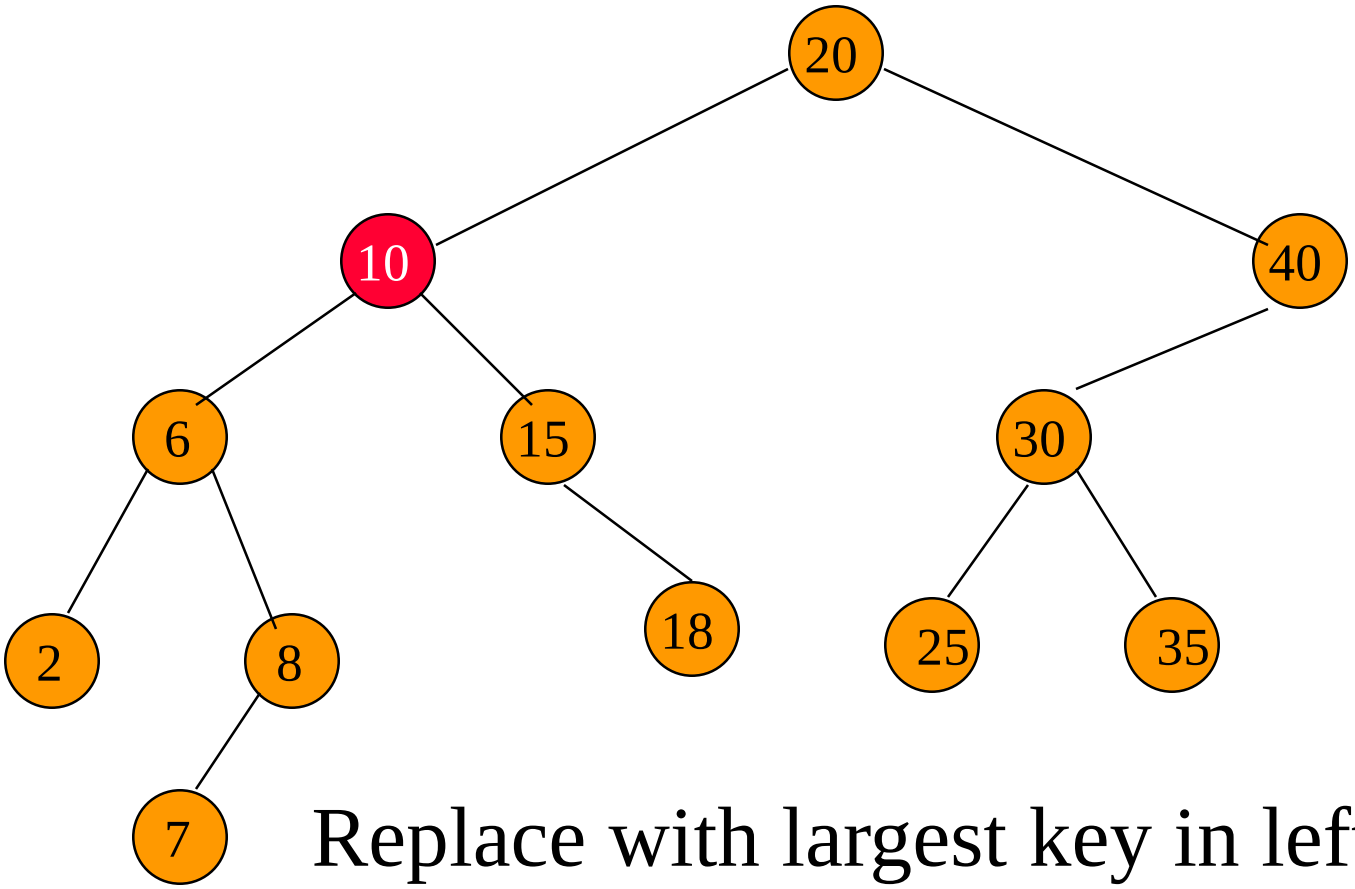
Remove from a degree 1 node. key = 15

Remove From A Degree 2 Node



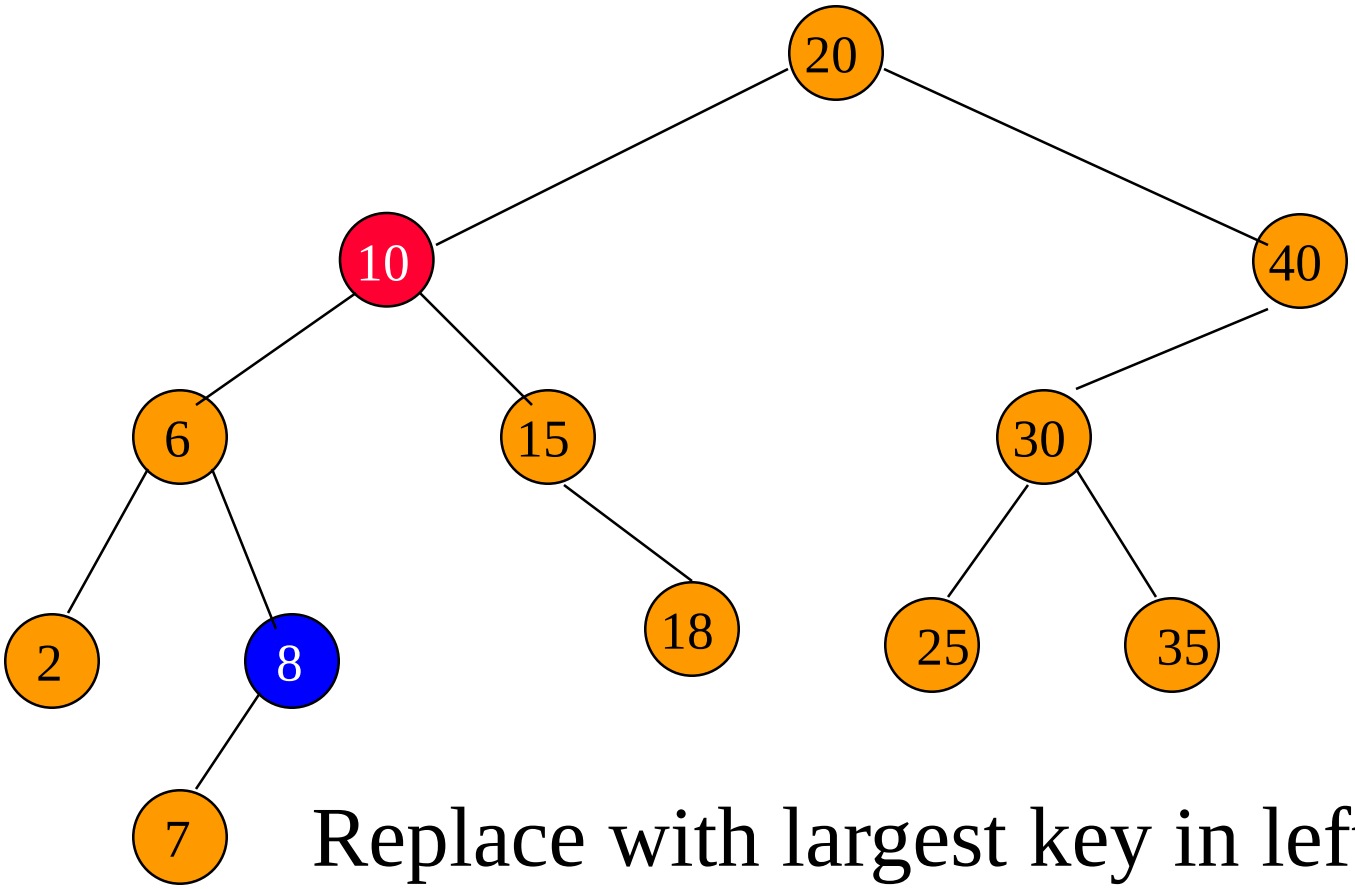
Remove from a degree 2 node. key = 10

Remove From A Degree 2 Node



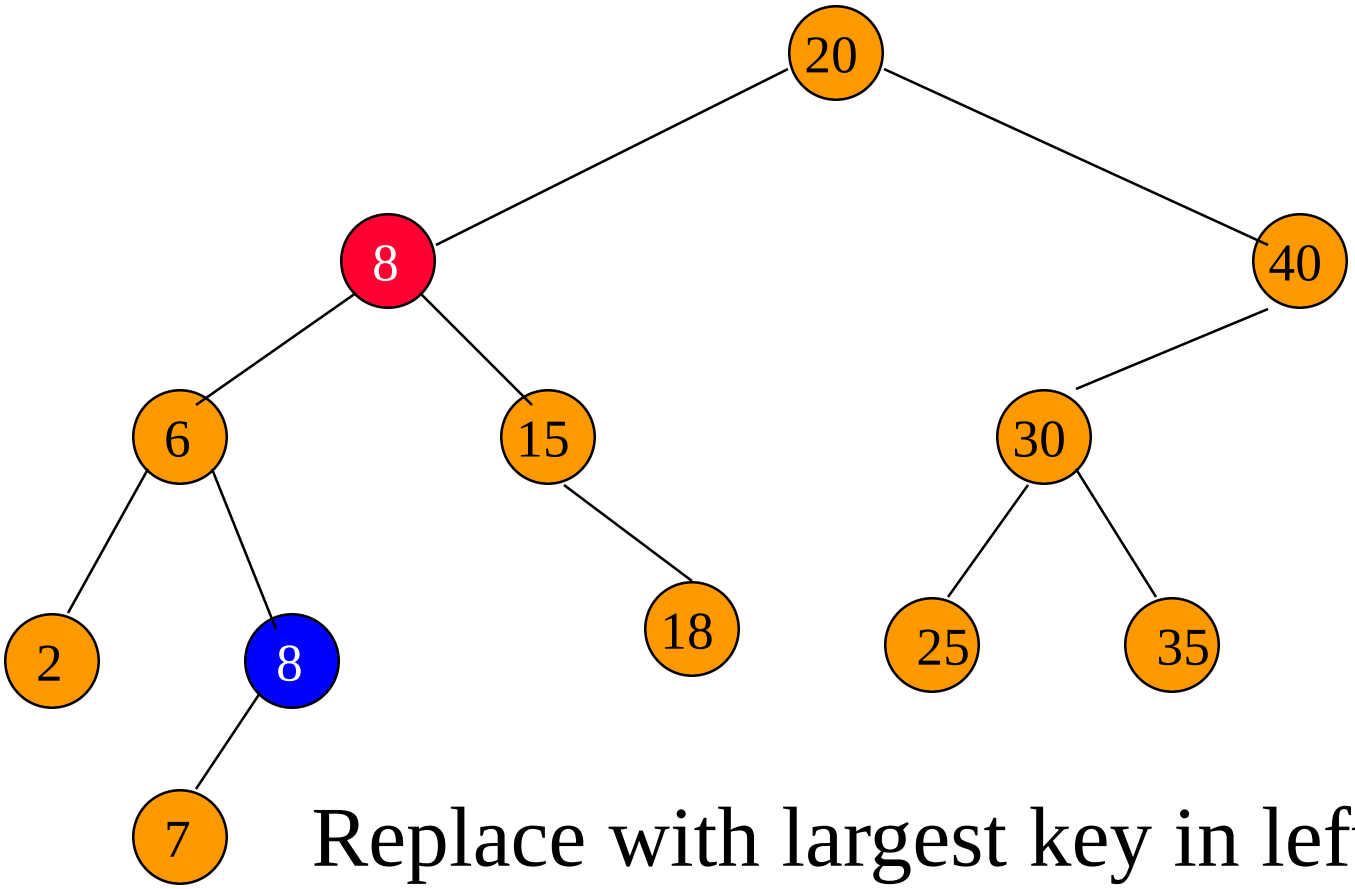
Replace with largest key in left subtree (or smallest in right subtree).

Remove From A Degree 2 Node



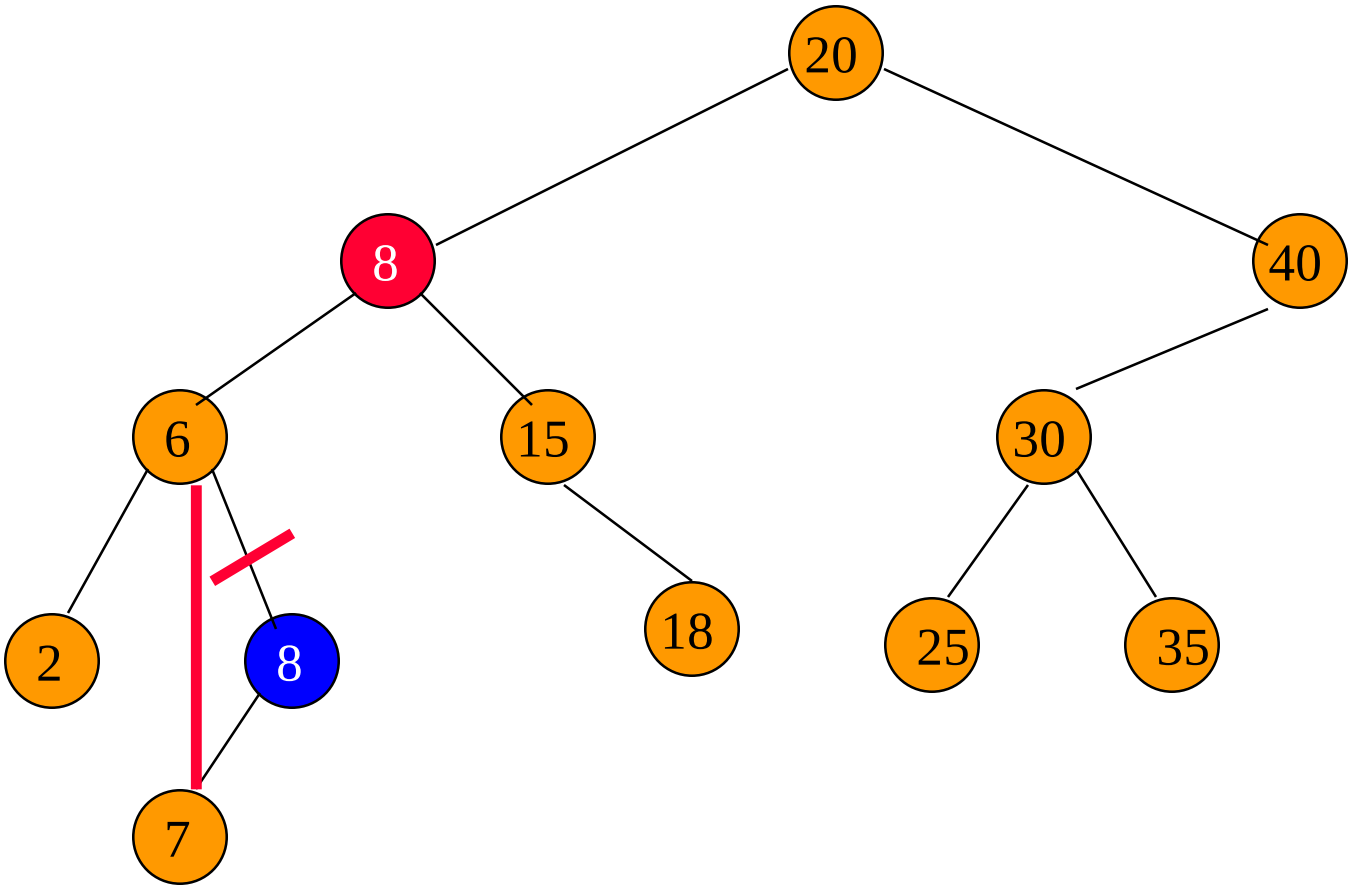
Replace with largest key in left subtree (or smallest in right subtree).

Remove From A Degree 2 Node



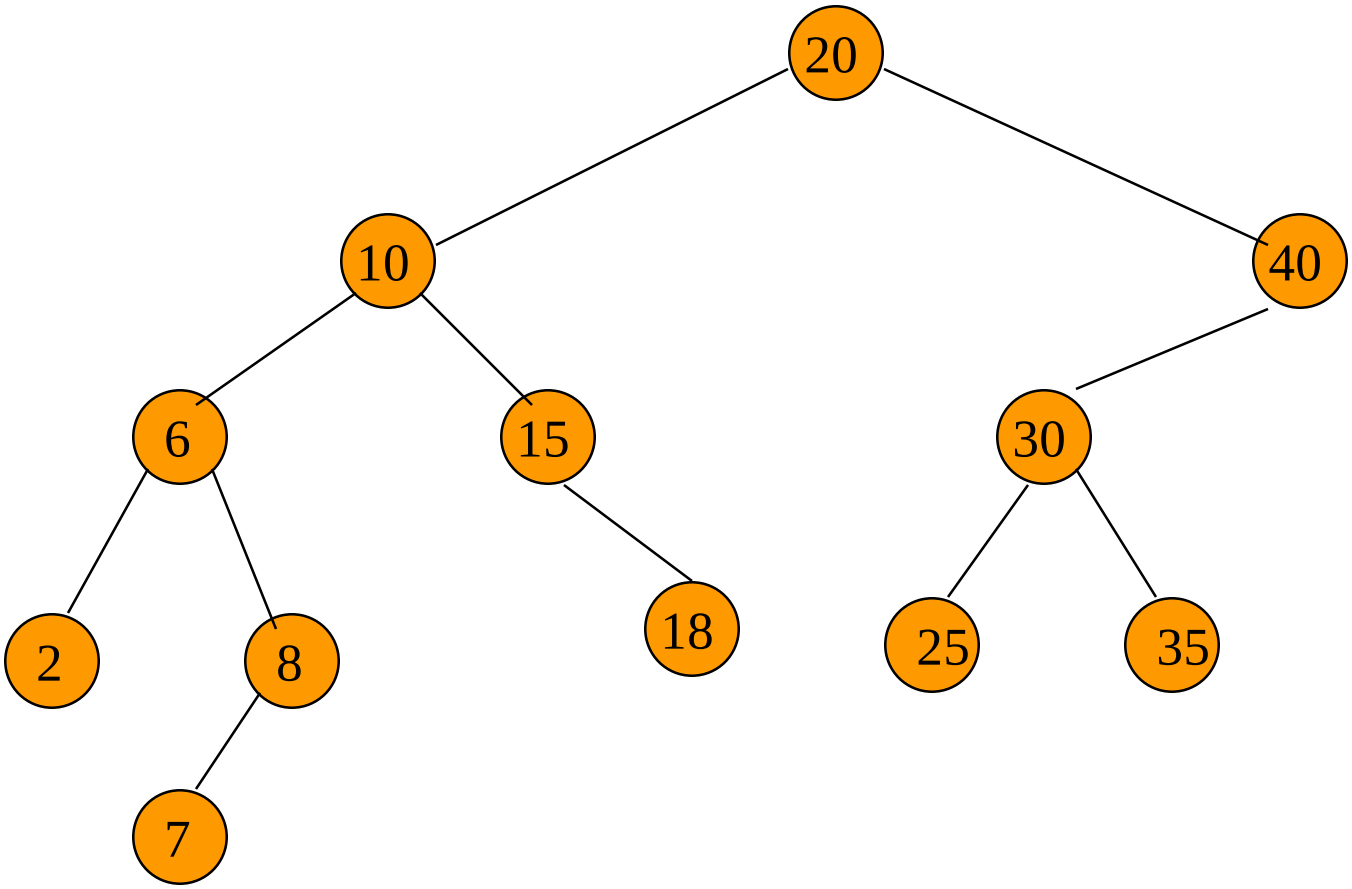
Replace with largest key in left subtree (or smallest in right subtree).

Remove From A Degree 2 Node



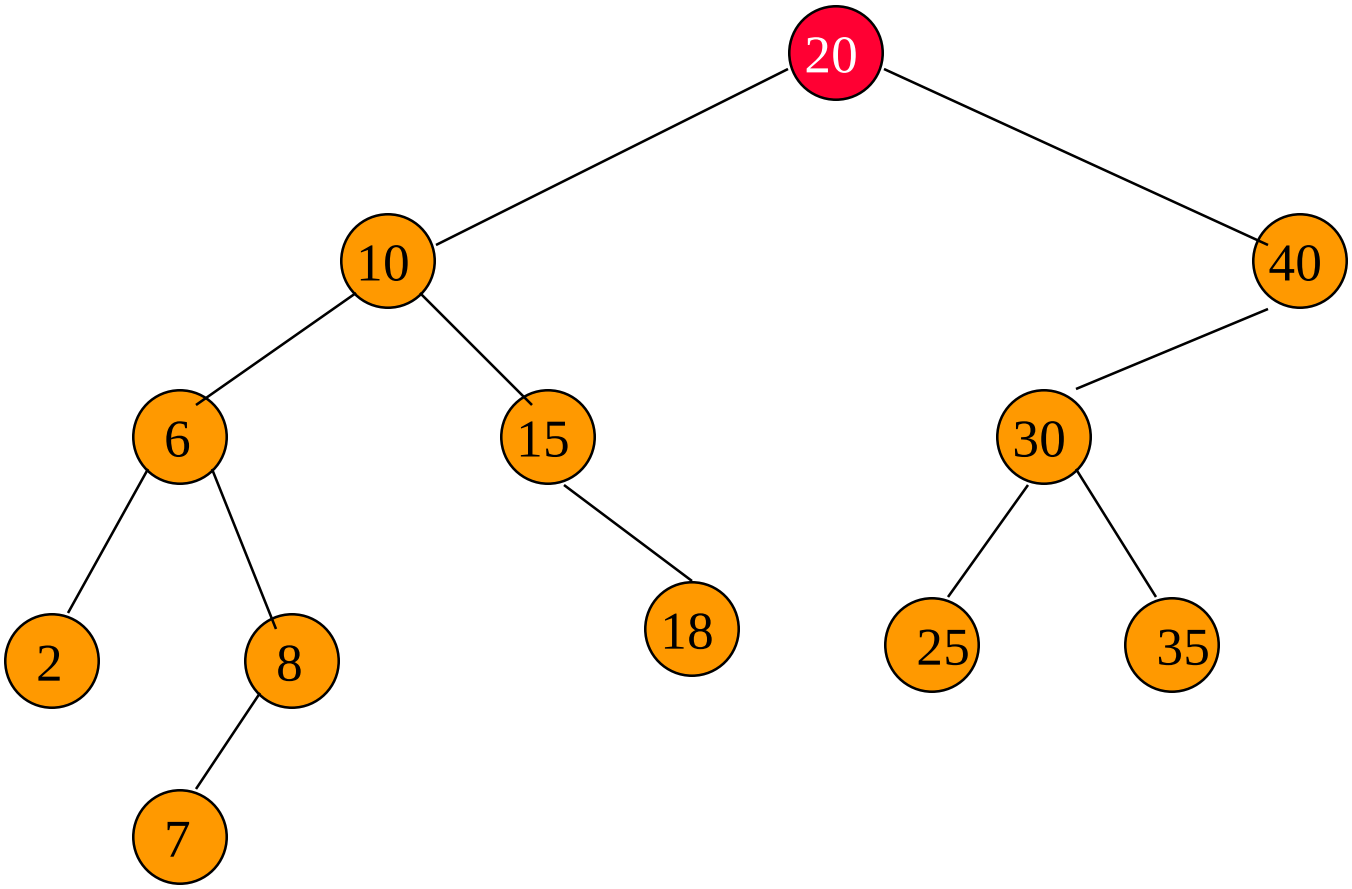
Largest key must be in a leaf or degree 1 node.

Another Remove From A Degree 2 Node



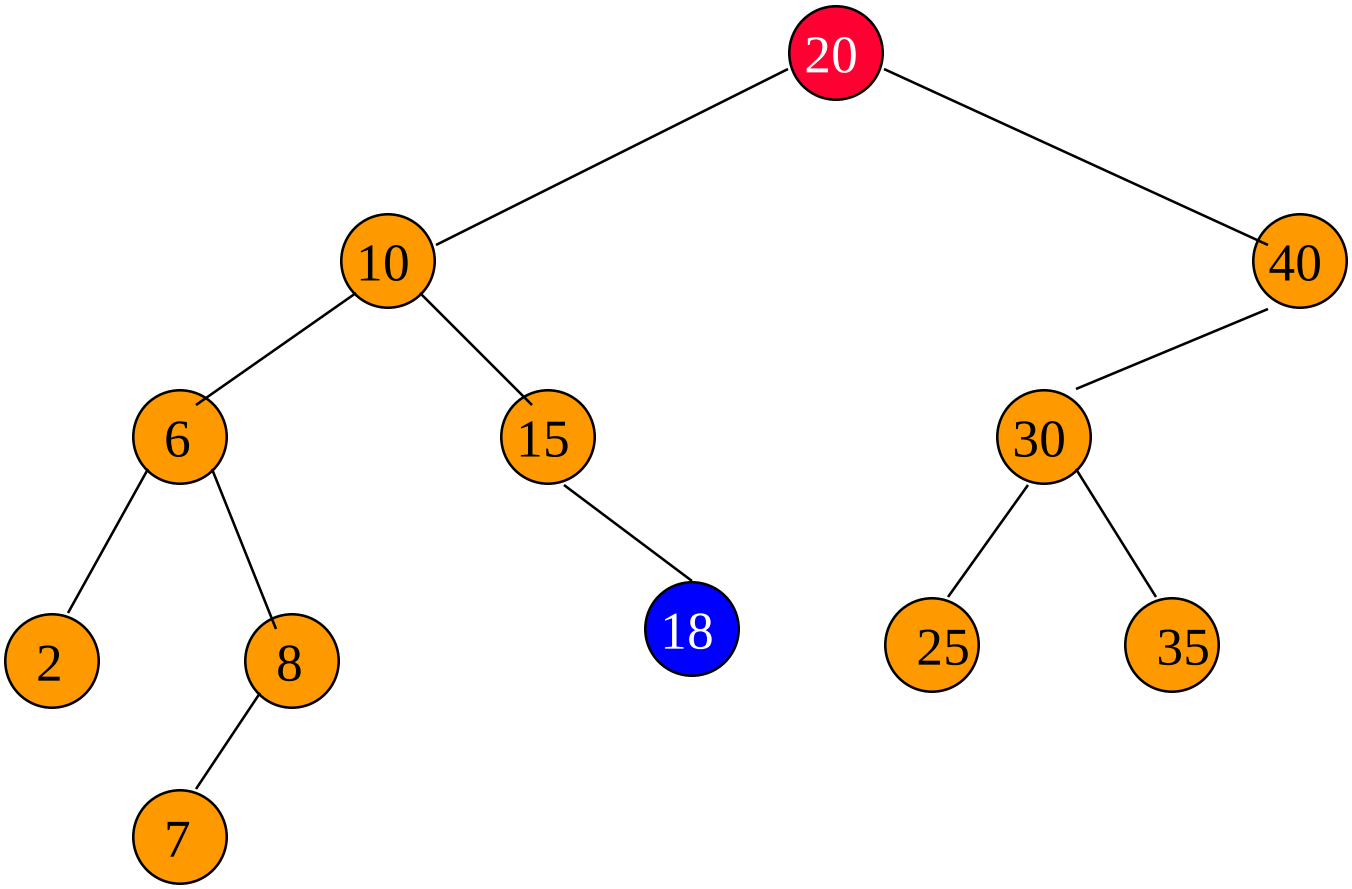
Remove from a degree 2 node. key = 20

Remove From A Degree 2 Node



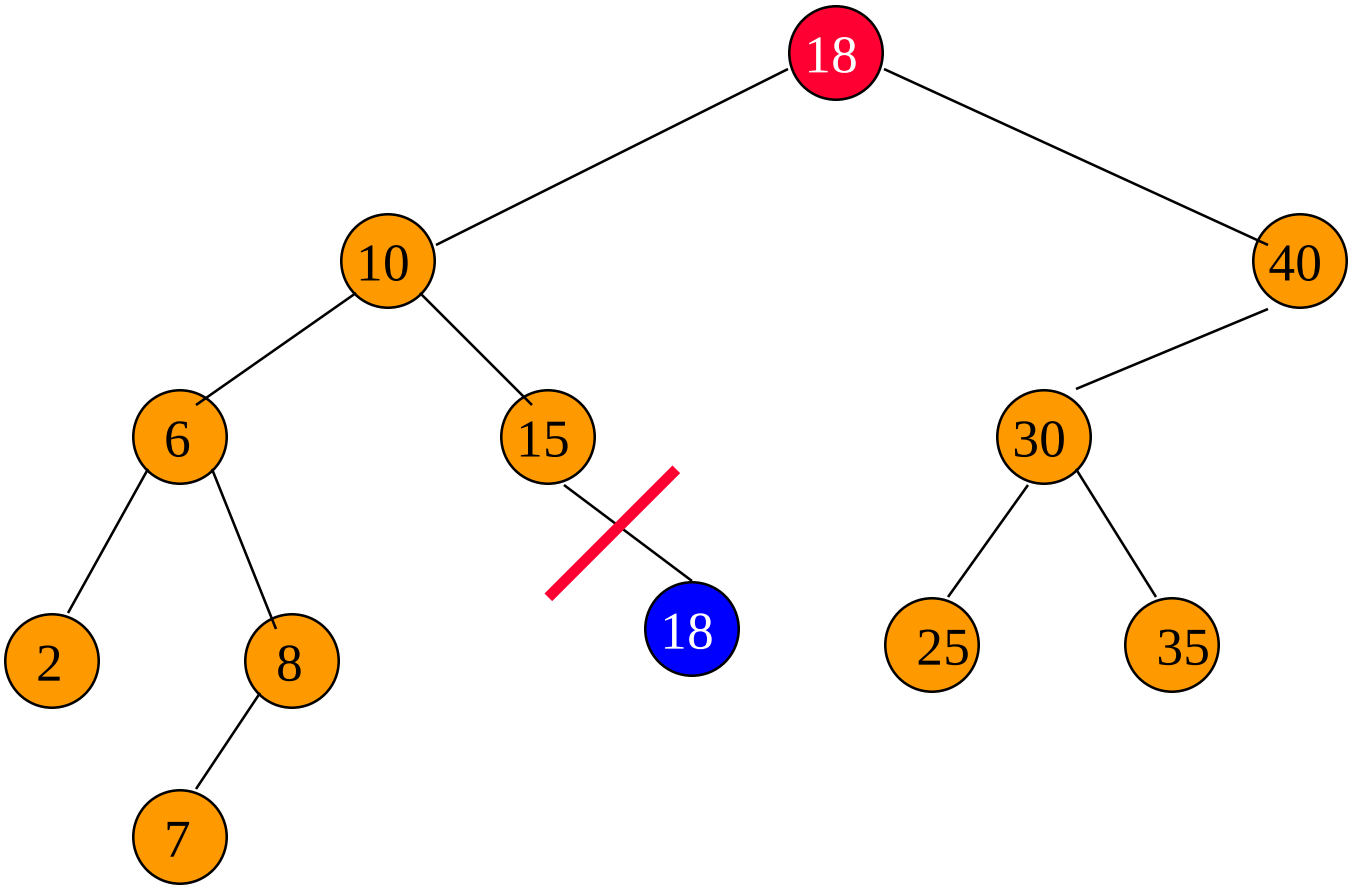
Replace with largest in left subtree.

Remove From A Degree 2 Node



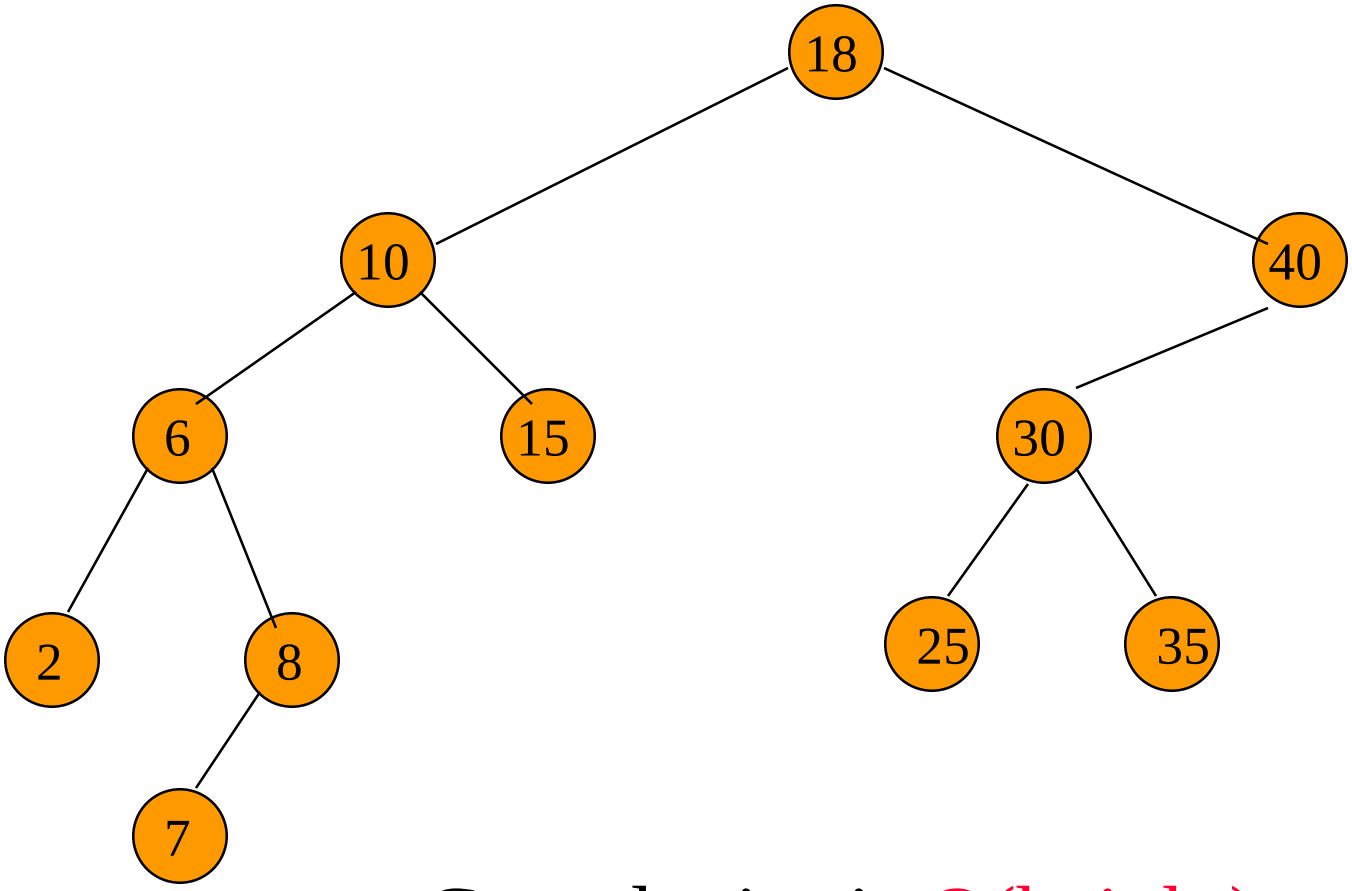
Replace with largest in left subtree.

Remove From A Degree 2 Node



Replace with largest in left subtree.

Remove From A Degree 2 Node

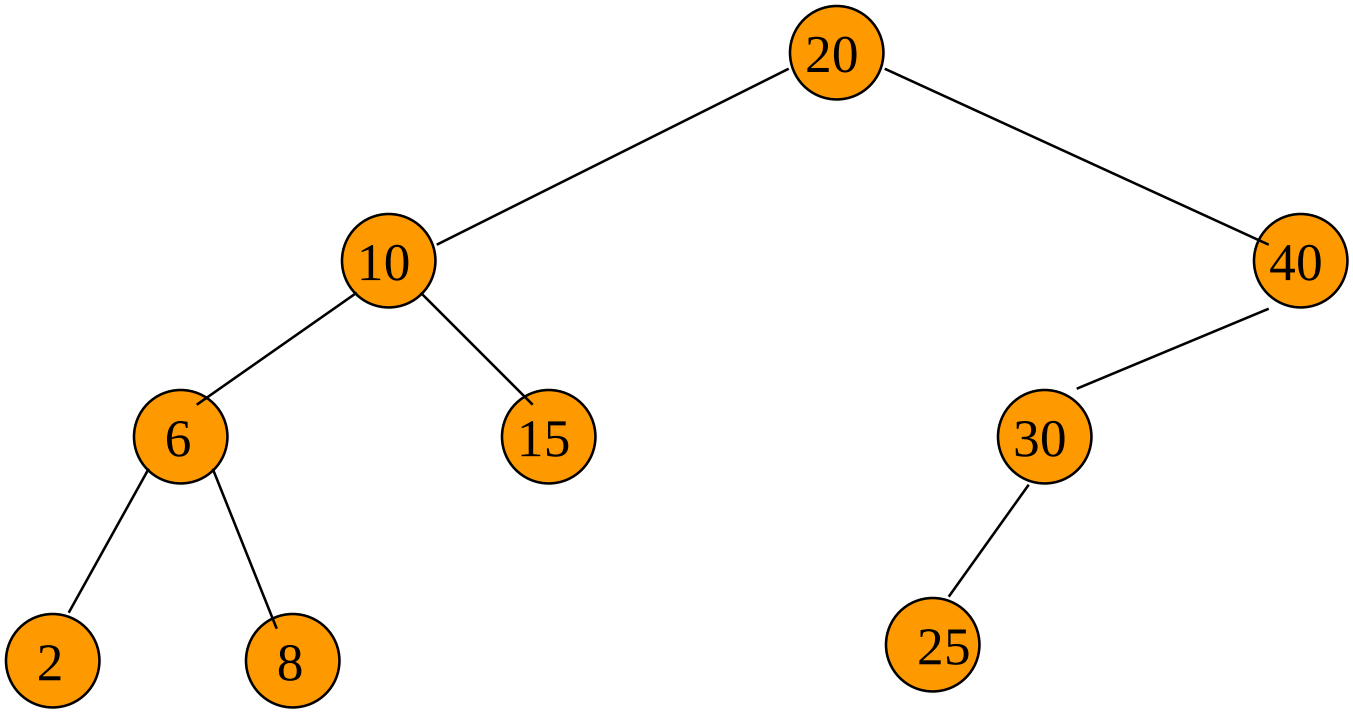


Complexity is O(height).

Yet Other Operations

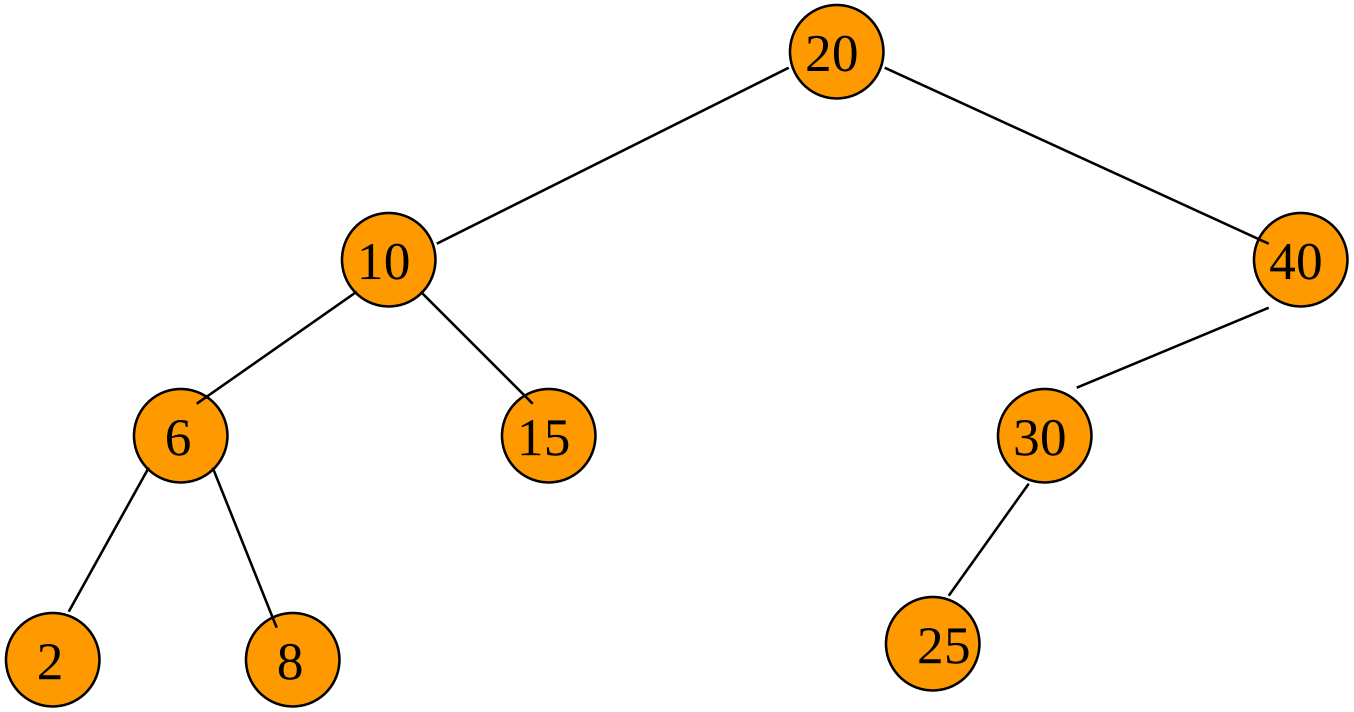
- Priority Queue Motivated Operations:
  - find max and/or min
  - remove max and/or min
  - initialize
  - meld

Max And/Or Min



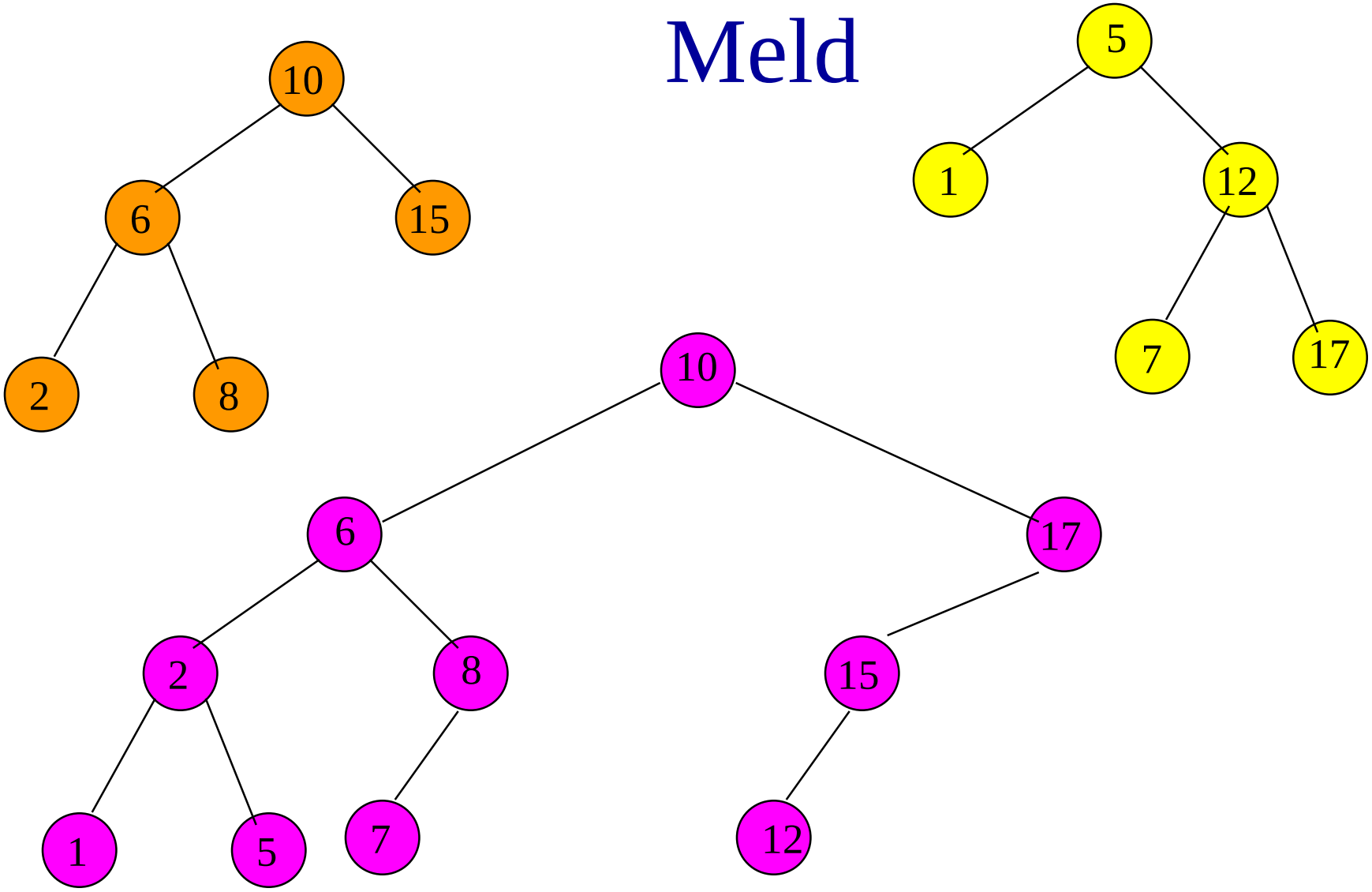
- Follow rightmost path to max element.
- Follow leftmost path to min element.
- Search and/or remove => O(h) time.

Initialize



- Sort n elements.
  - Initialize search tree.
  - Output in inorder (O(n)).
- Initialize must take O(n log n) time, because it isn't possible to sort faster than O(n log n).

Meld

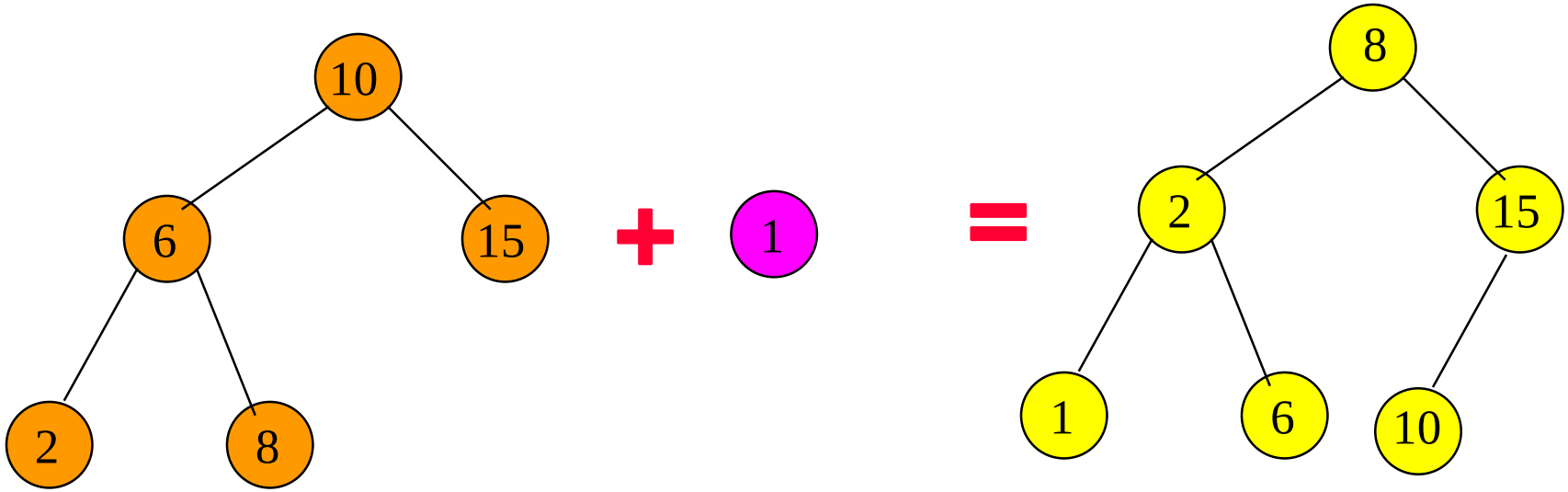


Meld And Merge

- Worst-case number of comparisons to merge two sorted lists of size n each is $2n-1$.
- So, complexity of melding two binary search trees of size n each is $\Omega(n)$.
- So, logarithmic time melding isn't possible.

O(log n) Height Trees

- Full binary trees.
  - Exist only when $n = 2^k - 1$.
- Complete binary trees.
  - Exist for all n.
  - Cannot insert/delete in $O(\log n)$ time.



Balanced Search Trees

- Height balanced.
  - ✓AVL (Adelson-Velsky and Landis) trees
- Weight Balanced.
- Degree Balanced.
  - ✓2-3 trees
  - ✓2-3-4 trees
  - ✓red-black trees
  - ✓B-trees

Acknowledgements

- E. Horowitz, S. Sahni, and D. Mehta, Fundamentals of data structures in C++, Second Edition, Silicon Press, 2007.
- Lecture Slides of Prof Sartaj Sahni.